LAPORAN AKHIR
PRAKTIKUM APLIKASI MOBILE

PENGEMBANGAN APLIKASI EXPENSE TRACKER BERBASIS FLUTTER

Disusun oleh:

HABIBI PUTRA RIZQULLAH

2411531001

Praktikum Kelas A

DOSEN PENGAMPU:

NURFIAH, S.ST, M.Kom

ASISTEN PRAKTIKUM:

EZZA ADDINI



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Allah Swt. atas kesempatan dan kelancaran dalam menyelesaikan praktikum Aplikasi Mobile serta pengembangan aplikasi Expense Tracker berbasis Flutter. Laporan ini disusun sebagai pertanggungjawaban atas penerapan materi praktikum pada aplikasi pencatatan keuangan yang dapat digunakan secara lokal.

Pengembangan dilakukan dengan menggabungkan source code pertemuan 01 sampai pertemuan 05. Widget, layout, form, dan navigasi yang telah dipelajari diterapkan pada satu alur aplikasi. Penyimpanan SQLite kemudian digunakan agar akun dan transaksi tetap tersedia ketika aplikasi dibuka kembali.

Penulis mengucapkan terima kasih kepada NURFIAH, S.ST, M.Kom selaku dosen pengampu dan EZZA ADDINI selaku asisten praktikum atas bimbingan selama kegiatan praktikum. Penulis juga berterima kasih kepada pihak yang membantu proses pembelajaran dan pemeriksaan aplikasi.

Laporan ini membahas penggabungan materi praktikum, implementasi fitur, penyimpanan data lokal, serta pengujian aplikasi berdasarkan dokumentasi yang tersedia. Penulis menyadari bahwa aplikasi dan laporan ini masih memiliki keterbatasan. Oleh karena itu, kritik dan saran diharapkan sebagai bahan perbaikan pada pengembangan berikutnya.

Padang, 24 Oktober 2026

HABIBI PUTRA RIZQULLAH
2411531001

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
DAFTAR GAMBAR iv
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 1
1.3 Tujuan Praktikum 1
BAB II LANDASAN TEORI 2
2.1 Flutter dan Dart 2
2.2 Widget dan Layout 2
2.3 Form dan Navigasi 2
2.4 Penyimpanan Lokal SQLite 3
BAB III PENGGABUNGAN DAN STRUKTUR APLIKASI 4
3.1 Gambaran Umum dan Struktur Proyek 4
3.2 Penerapan Materi Praktikum 1–5 4
3.3 Alur Navigasi dan Penyimpanan 5
BAB IV IMPLEMENTASI DAN PENGUJIAN 6
4.1 Lingkungan Pengembangan 6
4.2 Login dan Register 6
4.3 Validasi Input 7
4.4 Beranda, Riwayat, dan Bottom Navigation Bar 8
4.5 Profil, Email Pengguna, dan Logout 9
4.6 Form Transaksi dan Pemilihan Tanggal 10
4.7 Penyimpanan dan Detail Transaksi 11
4.8 SQLite, Saldo, dan Data Setelah Buka Ulang 12
4.9 Hasil Pengujian 13
4.10 Kode Program Lengkap per Langkah 14
4.10.1 Langkah 1 Inisialisasi dan Named Routes 15
4.10.2 Langkah 2 Halaman Loading 19
4.10.3 Langkah 3 Penyedia AppStore 21
4.10.4 Langkah 4 Model Transaksi 23
4.10.5 Langkah 5 Penyimpanan SQLite dan Akun 26
4.10.6 Langkah 6 Halaman Login 36
4.10.7 Langkah 7 Halaman Register 41
4.10.8 Langkah 8 Navigasi Utama 46

4.10.9 Langkah 9 Dashboard 49
4.10.10 Langkah 10 Kartu Saldo 53
4.10.11 Langkah 11 Tombol Aksi 56
4.10.12 Langkah 12 Riwayat Transaksi 58
4.10.13 Langkah 13 Item Transaksi 59
4.10.14 Langkah 14 Profil dan Logout 61
4.10.15 Langkah 15 Edit Profil 66
4.10.16 Langkah 16 Form Transaksi 70
4.10.17 Langkah 17 Detail Transaksi 81
4.10.18 Langkah 18 Kalkulator Kabataku 86
BAB V PENUTUP 92
5.1 Kesimpulan 92
DAFTAR PUSTAKA 93
LAMPIRAN 94
Lampiran A. Repository GitHub 94
Lampiran B. Dokumentasi Blog Praktikum 94
Lampiran C. Link Download APK 94
Lampiran D. Video Penjelasan Aplikasi 94
Lampiran E. Laporan Singkat 94
Lampiran F. Link Drive Semua 94

DAFTAR GAMBAR

Gambar 1. Halaman login aplikasi. 7
Gambar 2. Form pendaftaran akun lokal. 7
Gambar 3. Validasi email dan password kosong. 8
Gambar 4. Penolakan kredensial yang salah. 8
Gambar 5. Beranda dan kartu saldo pengguna. 9
Gambar 6. Riwayat transaksi milik akun aktif. 9
Gambar 7. Profil menampilkan nama dan email akun. 10
Gambar 8. Halaman login setelah logout. 10
Gambar 9. Input pengeluaran Beli Buku. 11
Gambar 10. Pemilihan tanggal transaksi. 11
Gambar 11. Notifikasi dan transaksi yang berhasil disimpan. 12
Gambar 12. Detail pengeluaran yang dipilih. 12
Gambar 13. Saldo setelah pengeluaran Rp 50.000. 13
Gambar 14. Data dan saldo tetap tersedia setelah buka ulang. 13
Gambar 15. Kode langkah 1 bagian 1. 15
Gambar 16. Kode langkah 1 bagian 2. 16
Gambar 17. Kode langkah 1 bagian 3. 17
Gambar 18. Kode langkah 1 bagian 4. 18
Gambar 19. Kode langkah 2 bagian 1. 19
Gambar 20. Kode langkah 2 bagian 2. 20
Gambar 21. Kode langkah 3 bagian 1. 21
Gambar 22. Kode langkah 3 bagian 2. 22
Gambar 23. Kode langkah 4 bagian 1. 23
Gambar 24. Kode langkah 4 bagian 2. 24
Gambar 25. Kode langkah 4 bagian 3. 25
Gambar 26. Kode langkah 5 bagian 1. 26
Gambar 27. Kode langkah 5 bagian 2. 27
Gambar 28. Kode langkah 5 bagian 3. 28
Gambar 29. Kode langkah 5 bagian 4. 29
Gambar 30. Kode langkah 5 bagian 5. 30
Gambar 31. Kode langkah 5 bagian 6. 31
Gambar 32. Kode langkah 5 bagian 7. 32
Gambar 33. Kode langkah 5 bagian 8. 33
Gambar 34. Kode langkah 5 bagian 9. 34
Gambar 35. Kode langkah 5 bagian 10. 35

Gambar 36. Kode langkah 6 bagian 1. 36
Gambar 37. Kode langkah 6 bagian 2. 37
Gambar 38. Kode langkah 6 bagian 3. 38
Gambar 39. Kode langkah 6 bagian 4. 39
Gambar 40. Kode langkah 6 bagian 5. 40
Gambar 41. Kode langkah 7 bagian 1. 41
Gambar 42. Kode langkah 7 bagian 2. 42
Gambar 43. Kode langkah 7 bagian 3. 43
Gambar 44. Kode langkah 7 bagian 4. 44
Gambar 45. Kode langkah 7 bagian 5. 45
Gambar 46. Kode langkah 8 bagian 1. 46
Gambar 47. Kode langkah 8 bagian 2. 47
Gambar 48. Kode langkah 8 bagian 3. 48
Gambar 49. Kode langkah 9 bagian 1. 49
Gambar 50. Kode langkah 9 bagian 2. 50
Gambar 51. Kode langkah 9 bagian 3. 51
Gambar 52. Kode langkah 9 bagian 4. 52
Gambar 53. Kode langkah 10 bagian 1. 53
Gambar 54. Kode langkah 10 bagian 2. 54
Gambar 55. Kode langkah 10 bagian 3. 55
Gambar 56. Kode langkah 11 bagian 1. 56
Gambar 57. Kode langkah 11 bagian 2. 57
Gambar 58. Kode langkah 12 bagian 1. 58
Gambar 59. Kode langkah 13 bagian 1. 59
Gambar 60. Kode langkah 13 bagian 2. 60
Gambar 61. Kode langkah 14 bagian 1. 61
Gambar 62. Kode langkah 14 bagian 2. 62
Gambar 63. Kode langkah 14 bagian 3. 63
Gambar 64. Kode langkah 14 bagian 4. 64
Gambar 65. Kode langkah 14 bagian 5. 65
Gambar 66. Kode langkah 15 bagian 1. 66
Gambar 67. Kode langkah 15 bagian 2. 67
Gambar 68. Kode langkah 15 bagian 3. 68
Gambar 69. Kode langkah 15 bagian 4. 69
Gambar 70. Kode langkah 16 bagian 1. 70
Gambar 71. Kode langkah 16 bagian 2. 71
Gambar 72. Kode langkah 16 bagian 3. 72

Gambar 73. Kode langkah 16 bagian 4. 73
Gambar 74. Kode langkah 16 bagian 5. 74
Gambar 75. Kode langkah 16 bagian 6. 75
Gambar 76. Kode langkah 16 bagian 7. 76
Gambar 77. Kode langkah 16 bagian 8. 77
Gambar 78. Kode langkah 16 bagian 9. 78
Gambar 79. Kode langkah 16 bagian 10. 79
Gambar 80. Kode langkah 16 bagian 11. 80
Gambar 81. Kode langkah 17 bagian 1. 81
Gambar 82. Kode langkah 17 bagian 2. 82
Gambar 83. Kode langkah 17 bagian 3. 83
Gambar 84. Kode langkah 17 bagian 4. 84
Gambar 85. Kode langkah 17 bagian 5. 85
Gambar 86. Kode langkah 18 bagian 1. 86
Gambar 87. Kode langkah 18 bagian 2. 87
Gambar 88. Kode langkah 18 bagian 3. 88
Gambar 89. Kode langkah 18 bagian 4. 89
Gambar 90. Kode langkah 18 bagian 5. 90
Gambar 91. Kode langkah 18 bagian 6. 91

BAB I
PENDAHULUAN

1.1 Latar Belakang

Pencatatan pemasukan dan pengeluaran diperlukan untuk mengetahui penggunaan uang serta memantau saldo yang tersedia. Catatan yang tidak tersusun dengan baik dapat menyulitkan pengguna saat memeriksa kembali transaksi yang telah dilakukan. Oleh karena itu, diperlukan aplikasi yang dapat membantu pengguna menyimpan dan melihat catatan keuangan secara teratur.

Expense Tracker dikembangkan untuk mencatat pemasukan, pengeluaran, dan saldo pada perangkat Android. Data disimpan secara lokal menggunakan SQLite agar catatan keuangan tetap tersedia ketika aplikasi ditutup dan dibuka kembali. Pengguna dapat menambahkan transaksi, melihat riwayat, serta memeriksa detail transaksi yang telah tersimpan.

Pengembangan aplikasi ini menggabungkan materi Praktikum Aplikasi Mobile pertemuan 1 sampai 5, meliputi penggunaan widget, penyusunan layout, pengelolaan form, validasi input, dan navigasi antarhalaman. Materi tersebut diterapkan dalam satu aplikasi agar setiap fitur saling terhubung dan mendukung alur pencatatan keuangan.

1.2 Rumusan Masalah

Permasalahan yang dibahas adalah bagaimana menggabungkan hasil praktikum dalam satu alur pencatatan keuangan, memvalidasi input, menyimpan transaksi secara lokal, dan menampilkan saldo yang sesuai dengan data pengguna yang sedang login.

1.3 Tujuan Praktikum

Tujuan pengembangan adalah menerapkan materi Praktikum 1–5, menyediakan register dan login lokal, menyimpan transaksi per akun, serta menghubungkan beranda, riwayat, form, dan detail transaksi. Aplikasi juga diperiksa melalui pengujian otomatis dan emulator, kemudian dibangun menjadi APK Android.

BAB II
LANDASAN TEORI

2.1 Flutter dan Dart

Flutter digunakan untuk membangun antarmuka aplikasi, sedangkan Dart digunakan untuk menulis logika program. Susunan dasar aplikasi terdiri atas MaterialApp, Scaffold, dan widget yang membentuk tampilan [1]. Pada praktikum ini, aplikasi dikembangkan dan diuji pada Android.

Nominal rupiah menggunakan tipe int agar perhitungan saldo memakai bilangan bulat. Operasi database, login, dan penyimpanan transaksi dijalankan dengan Future dan async/await. Pemeriksaan mounted dilakukan sebelum memperbarui tampilan setelah proses tersebut selesai.

2.2 Widget dan Layout

StatelessWidget digunakan untuk tampilan yang mengikuti data yang diterima, seperti halaman profil. StatefulWidget digunakan pada bagian yang memiliki perubahan keadaan, seperti form login, tab aktif, dan tombol untuk menampilkan atau menyembunyikan saldo. Perubahan tampilan dilakukan melalui setState(), sesuai materi pengenalan widget [1].

Row menyusun tombol secara horizontal, sedangkan Column menyusun informasi secara vertikal. Card digunakan untuk kartu saldo, ListTile untuk ringkasan transaksi, dan SingleChildScrollView agar halaman dapat digulir ketika isinya melebihi tinggi layar. Susunan ini mengacu pada materi layout dan styling [2].

2.3 Form dan Navigasi

Form digunakan untuk mengelompokkan kolom input dan menjalankan validasi melalui GlobalKey<FormState>. TextEditingController membaca nilai yang dimasukkan pengguna, sedangkan validator mengembalikan pesan jika input belum sesuai [3], [4]. Pada aplikasi, pola ini digunakan pada login, pendaftaran, edit profil, saldo awal, dan pencatatan transaksi.

Navigator.push membuka halaman baru, sedangkan Navigator.pop kembali ke halaman sebelumnya. Data transaksi dikirim ke halaman detail melalui constructor [5]. Named routes digunakan untuk mendaftarkan tujuan navigasi, dan BottomNavigationBar digunakan untuk memilih Beranda, Riwayat, atau Profil [6]. Ketiga tab ditempatkan dalam IndexedStack agar pergantian tab tidak menambah halaman pada riwayat navigasi.

2.4 Penyimpanan Lokal SQLite

SQLite digunakan untuk menyimpan data aplikasi secara lokal pada perangkat Android. Data yang disimpan meliputi akun pengguna, transaksi, sesi login, dan isian form yang belum selesai. Setiap transaksi dihubungkan dengan akun pemiliknya agar pengguna hanya dapat mengakses catatan keuangannya sendiri. Data tersebut tetap tersimpan ketika aplikasi ditutup dan dapat digunakan kembali saat aplikasi dibuka.

Saat mendaftar, password pengguna diolah menjadi hash sebelum disimpan ke database. Ketika pengguna login, aplikasi memeriksa kecocokan password melalui proses verifikasi hash. Proses pendaftaran dan login dilakukan langsung pada perangkat sehingga tidak memerlukan koneksi internet.

BAB III
PENGGABUNGAN DAN STRUKTUR APLIKASI

3.1 Gambaran Umum dan Struktur Proyek

Expense Tracker menyediakan pendaftaran akun, login, beranda, riwayat, profil, pencatatan transaksi, dan detail transaksi. Pengguna juga dapat mengedit atau menghapus transaksi, mengatur saldo awal, serta membuka Kalkulator Kabataku. Kode aplikasi gabungan disimpan dalam folder tersendiri sehingga file latihan praktikum tetap tersedia.

```
lib/
  pertemuan_01/ ... pertemuan_05/   (source asli)
  gabungan_praktikum/
    main.dart, main_screen.dart
    login_screen.dart, register_screen.dart
    dashboard_screen.dart, history_screen.dart
    profile_screen.dart, edit_profile_screen.dart
    add_transaction_screen.dart
    transaction_detail_screen.dart, storage.dart
    models/, widgets/, latihan/, assets/
```

Entry point gabungan adalah lib/gabungan_praktikum/main.dart. MyApp menyediakan MaterialApp, tema, dan routes. AppStore pada storage.dart menangani akses SQLite, sementara TransactionModel mengubah data tabel menjadi objek yang dipakai halaman. MainScreen memuat ulang daftar transaksi dan membagikannya kepada tab.

3.2 Penerapan Materi Praktikum 1–5

Folder pertemuan_01 berisi latihan TextField, TextFormField, dan Kalkulator Kabataku. Controller dan validasi form dari latihan tersebut diterapkan kembali pada input aplikasi, termasuk edit profil [3]. Kalkulator dapat dibuka dari Beranda dan form transaksi.

Folder pertemuan_02 berisi modul1_widgets dan modul2_layouts. Widget sapaan, kartu saldo interaktif, susunan tombol dengan Row, dan daftar transaksi dengan ListTile diterapkan pada aplikasi gabungan [1], [2]. Nilai saldo yang semula statis diganti dengan perhitungan dari data pengguna.

Folder pertemuan_03 menjadi dasar form transaksi, validasi, pilihan kategori, dan tanggal [4]. Materi navigasi dan pengiriman data pada pertemuan_04 diterapkan pada halaman detail transaksi [5]. Sementara itu, pertemuan_05

digunakan sebagai dasar named routes, tiga tab utama, dan logout [6]. Penomoran folder mengikuti proyek praktikum, sehingga berbeda dengan nomor yang tercantum pada sebagian modul. Login simulasi kemudian dikembangkan menjadi login menggunakan akun yang tersimpan di SQLite.

3.3 Alur Navigasi dan Penyimpanan

Saat aplikasi dibuka, _SessionGate memulihkan sesi. Sesi yang valid membuka MainScreen; sesi kosong membuka LoginScreen. Setelah login, pengguna berpindah tab tanpa menumpuk route. Form dibuka melalui /add, kalkulator melalui /calculator, dan edit akun melalui /edit-profile. Detail dibuka dengan Navigator.push dan menerima TransactionModel.

Database gabungan_praktikum.db berisi users untuk identitas dan saldo awal, transactions untuk catatan keuangan, session untuk satu sesi aktif, serta drafts untuk input yang belum selesai. Transactions memiliki user_id sebagai foreign key. Pembacaan, perubahan, dan penghapusan transaksi dibatasi oleh user_id akun aktif. Logout hanya menghapus sesi, sehingga akun maupun transaksi tetap tersimpan.

BAB IV
IMPLEMENTASI DAN PENGUJIAN

4.1 Lingkungan Pengembangan

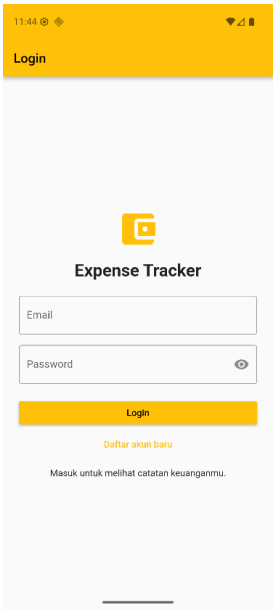
Lingkungan pengembangan menggunakan Windows, Flutter 3.47.2, Dart 3.13.2, Java 17, AGP 9.1.0, dan Gradle 9.3.1. Pengujian memakai Pixel 9 (emulator-5554), Android 16/API 36, x86_64. Package utama adalah sqflite, path, dan cryptography.

Expense Tracker menggunakan application ID com.example.praktikum_1, versi 1.0.0+1, minimum SDK 24, dan target SDK 36. Perintah menjalankan entry point gabungan:

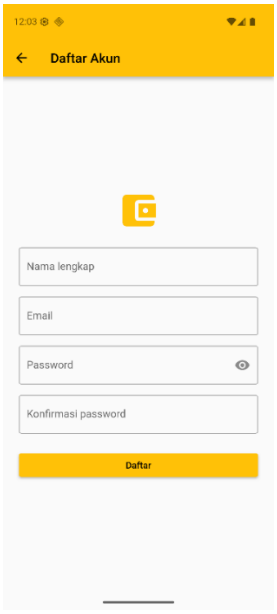
```
flutter run -d emulator-5554
  -t lib/gabungan_praktikum/main.dart
```

4.2 Login dan Register

Pendaftaran akun meminta nama lengkap, email, password, dan konfirmasi password. AppStore menyeragamkan penulisan email, memeriksa apakah email sudah terdaftar, lalu menyimpan identitas akun beserta hash dan salt password. Setelah pendaftaran berhasil, pengguna kembali ke halaman login. Login memeriksa kecocokan email dan password dengan data akun yang tersimpan. Nama akun ditampilkan pada Beranda dan Profil.



Gambar 1. Halaman login aplikasi.



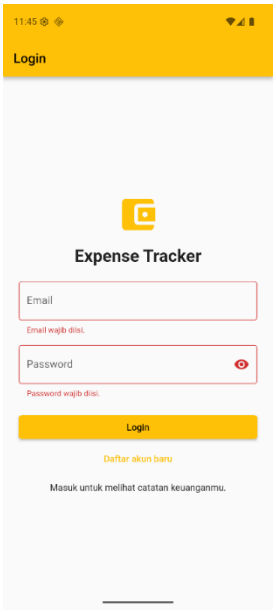
Gambar 2. Form pendaftaran akun lokal.

Kode lengkap untuk fitur ini disajikan pada Bagian 4.10.

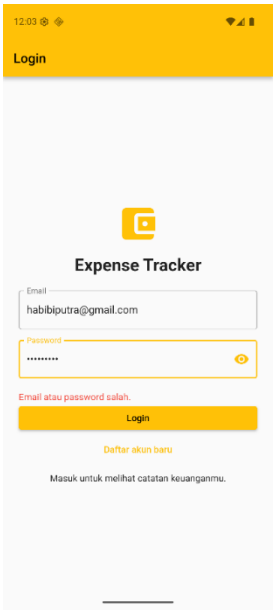
4.3 Validasi Input

Materi Form dan TextFormField diterapkan pada field yang wajib diisi. Login menolak email atau password kosong. Register memeriksa format email, panjang password minimal delapan karakter, serta kesamaan konfirmasi password. Pada transaksi, judul wajib diisi, nominal harus berupa bilangan bulat positif, kategori harus terdaftar, dan tanggal harus dipilih.

Ketika tombol login ditekan tanpa mengisi email dan password, kedua kolom menampilkan pesan kesalahan. Jika password salah, aplikasi menampilkan pesan “Email atau password salah.” dan tetap berada pada halaman login. Hasil pengujian tersebut ditunjukkan pada screenshot berikut.



Gambar 3. Validasi email dan password kosong.



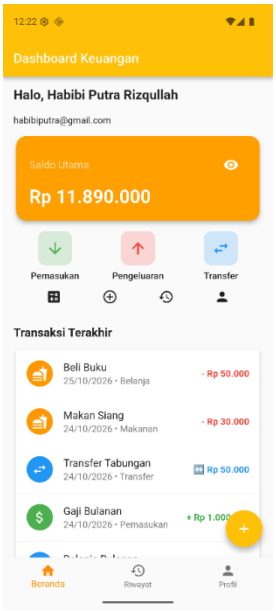
Gambar 4. Penolakan kredensial yang salah.

Kode lengkap untuk fitur ini disajikan pada Bagian 4.10.

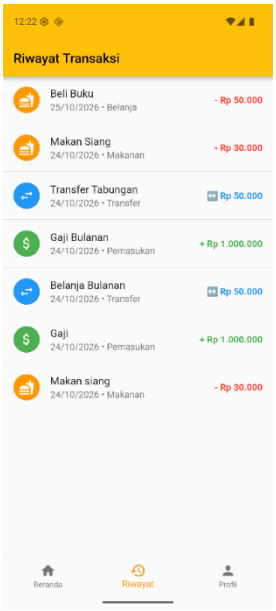
4.4 Beranda, Riwayat, dan Bottom Navigation Bar

Beranda menampilkan nama dan email akun, kartu saldo, tombol aksi, serta transaksi tersimpan. Saldo dapat disembunyikan melalui ikon mata. Riwayat membaca daftar yang sama dan menampilkan judul, kategori, tanggal, serta nominal. Nilai terbaru dimuat kembali melalui callback setelah form atau detail ditutup.

MainScreen menyimpan indeks tab yang sedang dipilih. Saat pengguna menekan BottomNavigationBar, indeks pada IndexedStack berubah untuk menampilkan Beranda, Riwayat, atau Profil. Susunan navigasi mengikuti modul arsitektur navigasi [6], sedangkan komponen tampilannya menerapkan materi layout [2].



Gambar 5. Beranda dan kartu saldo pengguna.



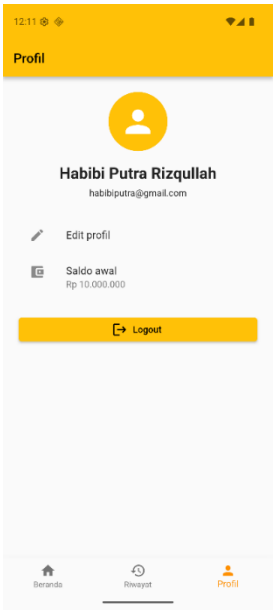
Gambar 6. Riwayat transaksi milik akun aktif.

Kode lengkap untuk fitur ini disajikan pada Bagian 4.10.

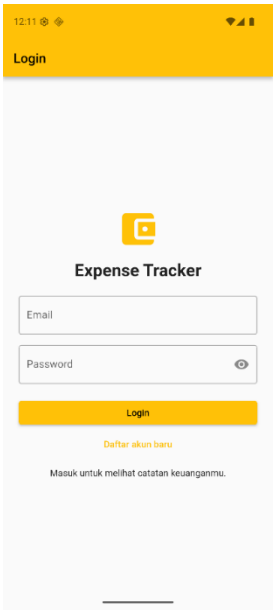
4.5 Profil, Email Pengguna, dan Logout

Profil menampilkan nama hasil register dan email dari akun aktif. Menu yang tersedia adalah Edit profil, Saldo awal, dan Logout. Edit profil melanjutkan pola form Pertemuan 1 dan menyimpan perubahan pada baris users yang sama. Saldo awal menerima angka nol atau positif dan ikut digunakan dalam perhitungan saldo.

Logout memanggil AppStore.logout(), kemudian menghapus stack navigasi melalui pushNamedAndRemoveUntil. Database tetap dipertahankan. Pada pemeriksaan emulator, tombol kembali setelah logout tidak membuka halaman akun sebelumnya; ketika aplikasi dibuka kembali, halaman login ditampilkan.



Gambar 7. Profil menampilkan nama dan email akun.



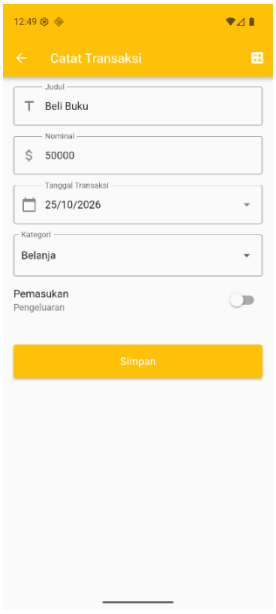
Gambar 8. Halaman login setelah logout.

Kode lengkap untuk fitur ini disajikan pada Bagian 4.10.

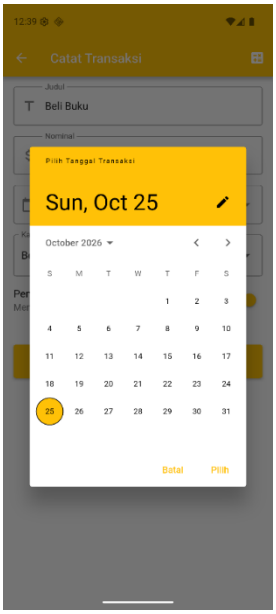
4.6 Form Transaksi dan Pemilihan Tanggal

Form melanjutkan kode transaksi Pertemuan 3 dan 5. Input mencakup judul, nominal, tanggal, kategori, serta jenis transaksi. Pada pengujian, dicatat pengeluaran “Beli Buku” sebesar Rp 50.000 dengan kategori Belanja dan tanggal 25/10/2026. Tanggal dipilih melalui showDatePicker, bukan diketik bebas.

Tanggal yang dipilih ditampilkan sebagai DD/MM/YYYY, lalu dikonversi bersama waktu pencatatan menjadi ISO 8601 UTC sebelum disimpan. Dropdown menyediakan kategori yang diizinkan. Draft input tersimpan per akun, sedangkan draft transaksi dibersihkan setelah penyimpanan berhasil.



Gambar 9. Input pengeluaran Beli Buku.



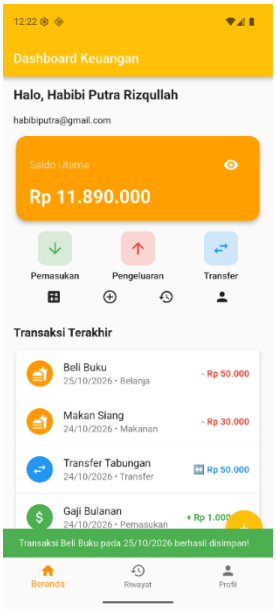
Gambar 10. Pemilihan tanggal transaksi.

Kode lengkap untuk fitur ini disajikan pada Bagian 4.10.

4.7 Penyimpanan dan Detail Transaksi

Saat tombol Simpan ditekan, aplikasi memvalidasi input dan membentuk TransactionModel. AppStore.saveTransaction() menambahkan transaksi baru atau memperbarui transaksi berdasarkan id dan user_id. Setelah transaksi tersimpan, form ditutup dan daftar transaksi diperbarui. Pada screenshot, pengeluaran Beli Buku sebesar Rp 50.000 sudah tercatat.

Saat transaksi dipilih, aplikasi membuka halaman detail melalui MaterialPageRoute dan mengirim objek transaksi melalui constructor [5]. Halaman ini menampilkan judul, nominal, kategori, tanggal, dan jenis transaksi. Tombol Edit membuka form dengan data sebelumnya, sedangkan tombol Hapus meminta konfirmasi sebelum transaksi dihapus.



Gambar 11. Notifikasi dan transaksi yang berhasil disimpan.



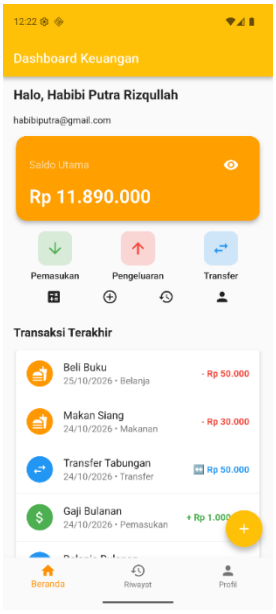
Gambar 12. Detail pengeluaran yang dipilih.

Kode lengkap untuk fitur ini disajikan pada Bagian 4.10.

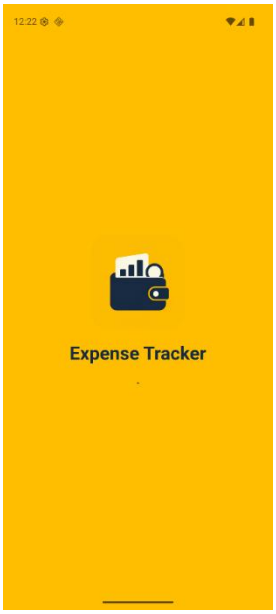
4.8 SQLite, Saldo, dan Data Setelah Buka Ulang

AppStore mengaktifkan foreign key pada pembukaan database. Query transaksi menggunakan parameter user_id agar data tiap akun terpisah. Tabel users menyimpan saldo awal, sedangkan transactions menyimpan nominal dan kind. Saldo dihitung dari saldo awal ditambah pemasukan, dikurangi pengeluaran. Catatan transfer bernilai netral karena aplikasi belum memiliki model rekening asal dan tujuan.

Sebelum Beli Buku disimpan, saldo pengujian Rp 11.940.000. Pengeluaran Rp 50.000 mengubahnya menjadi Rp 11.890.000. Aplikasi kemudian dihentikan dengan force-stop dan dibuka kembali. Nama, transaksi Beli Buku, sesi, dan saldo Rp 11.890.000 tetap tampil.



Gambar 13. Saldo setelah pengeluaran Rp 50.000.



Gambar 14. Data dan saldo tetap tersedia setelah buka ulang.

Kode lengkap untuk fitur ini disajikan pada Bagian 4.10.

4.9 Hasil Pengujian

Pengujian meliputi analisis kode, pengujian unit dan widget, serta pemeriksaan langsung pada emulator. Perintah flutter analyze lib/gabungan_praktikum menghasilkan “No issues found!”. Empat pengujian unit dan widget juga lulus. Hasil pengujian disimpan dalam folder pengujian dan dirangkum sebagai berikut.

1. Register dan login: pengujian SQLite memeriksa pendaftaran akun, normalisasi email, penolakan email duplikat, login salah, serta login benar. Pengujian widget memeriksa form kosong, format email, dan konfirmasi password yang berbeda. Hasilnya sesuai dengan validasi yang diimplementasikan.

2. Transaksi dan saldo: pengujian SQLite memeriksa tambah, baca, edit, dan hapus transaksi, nominal nol yang ditolak, serta transfer yang tidak menambah atau mengurangi saldo. Pada skenario terpisah, saldo awal Rp 50.000, pemasukan Rp 100.000, dan pengeluaran Rp 25.000 menghasilkan Rp 125.000. Setelah pengeluaran diedit menjadi Rp 30.000, saldo menjadi Rp 120.000.

3. Pemisahan akun dan penyimpanan data: koneksi SQLite ditutup lalu dibuka kembali untuk memeriksa data dan sesi login. Akun kedua tidak dapat

membaca transaksi, detail, atau draft milik akun pertama. Percobaan mengubah dan menghapus transaksi milik akun lain juga ditolak. Data akun pertama tetap tersedia ketika pengguna login kembali.

4. Pengujian pada emulator: sembilan skenario berhasil dijalankan, meliputi login kosong, password salah, login benar, riwayat transaksi, tambah pengeluaran, detail transaksi, profil, logout dan tombol kembali, serta menutup paksa dan membuka ulang aplikasi. Saldo dan judul transaksi dicatat dalam screenshot dan berkas emulator-laporan.json.

5. Pengujian integrasi: log pengembangan pada Pixel 9 mencatat satu pengujian integrasi yang lulus. Skenarionya mencakup konfirmasi password berbeda, email duplikat, pencatatan dan perubahan transaksi, profil, kalkulator, pemisahan akun, serta logout. Hasil ini menggunakan log pengembangan yang telah tersedia; pengujian tidak dijalankan ulang saat pengambilan screenshot laporan.

6. Tampilan: pengujian widget dilakukan pada lebar layar 320 piksel logis dan skala teks 1,5, termasuk saat saldo ditampilkan dan disembunyikan. Pada pengujian tersebut, tidak ditemukan tampilan yang melewati batas layar. Ukuran perangkat lain belum diuji.

4.10 Kode Program Lengkap per Langkah

Bagian ini memuat seluruh kode Dart pada folder gabungan_praktikum. Setiap file disajikan sebagai satu langkah. File yang panjang dibagi menjadi beberapa gambar berurutan dengan rentang baris sumber yang saling berlanjut. Tidak ada baris kode yang dihilangkan.

4.10.1 Langkah 1 Inisialisasi dan Named Routes

Mendaftarkan halaman, mengatur tema, dan menentukan halaman awal berdasarkan sesi pengguna.

File sumber: lib/gabungan_praktikum/main.dart

Bagian 1, baris 1 sampai 25.

Langkah 1 Bagian 1

```dart
import 'package:flutter/material.dart';

import 'main_screen.dart';
import 'login_screen.dart';
import 'register_screen.dart';
import 'add_transaction_screen.dart';
import 'storage.dart';
import 'store_scope.dart';
import 'loading_screen.dart';
import 'models/transaction_model.dart';
import 'latihan/kalkulator_kabataku.dart' as calculator;
import 'edit_profile_screen.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const MyApp());
}

class MyApp extends StatefulWidget {
  const MyApp({super.key, this.store});
  final AppStore? store;
  @override
  State<MyApp> createState() => _MyAppState();
}
```

Gambar 15. Kode langkah 1 bagian 1.

Bagian 2, baris 26 sampai 57.

Langkah 1 Bagian 2

```
class _MyAppState extends State<MyApp> {
  late final store = widget.store ?? AppStore();
  @override
  Widget build(BuildContext context) => StoreScope(
    store: store,
    child: MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Expense Tracker',
      theme: ThemeData(primarySwatch: Colors.amber, useMaterial3: false),
      initialRoute: '/',
      routes: {
        '/': (_) => _SessionGate(store: store),
        '/login': (_) => LoginScreen(store: store),
        '/register': (_) => RegisterScreen(store: store),
      },
      onGenerateRoute: (settings) => MaterialPageRoute<dynamic>(
        settings: settings,
        builder: (context) {
          if (store.user == null) return LoginScreen(store: store);
          switch (settings.name) {
            case '/main':
              return MainScreen(store: store);
            case '/add':
              return AddTransactionScreen(
                store: store,
                initialKind: settings.arguments is String
                    ? settings.arguments as String
                    : 'expense',
                transaction: settings.arguments is TransactionModel
                    ? settings.arguments as TransactionModel
                    : null,
              );
```

Gambar 16. Kode langkah 1 bagian 2.

Bagian 3, baris 58 sampai 87.

Langkah 1 Bagian 3

```
            case '/calculator':
              return _exercise(
                'Kalkulator Kabataku',
                const calculator.KalkulatorForm(),
              );
            case '/edit-profile':
              return EditProfileScreen(store: store);
            default:
              return MainScreen(store: store);
          }
        },
      ),
    ),
  );
  Widget _exercise(String title, Widget body) => Theme(
    data: ThemeData(),
    child: Scaffold(
      appBar: AppBar(title: Text(title)),
      body: SafeArea(child: SingleChildScrollView(child: body)),
    ),
  );
}

class _SessionGate extends StatefulWidget {
  final AppStore store;
  const _SessionGate({required this.store});
  @override
  State<_SessionGate> createState() => _SessionGateState();
}
```

Gambar 17. Kode langkah 1 bagian 3.

Bagian 4, baris 88 sampai 121.

Langkah 1 Bagian 4

```
class _SessionGateState extends State<_SessionGate> {
  Future<void> _initialize() async {
    await Future.wait([
      widget.store.restoreSession(),
      Future<void>.delayed(const Duration(milliseconds: 900)),
    ]);
  }

  late Future<void> session = _initialize();
  @override
  Widget build(BuildContext context) => FutureBuilder<void>(
    future: session,
    builder: (context, snapshot) {
      if (snapshot.connectionState != ConnectionState.done) {
        return const LoadingScreen();
      }
      if (snapshot.hasError) {
        return Scaffold(
          body: Center(
            child: TextButton(
              onPressed: () => setState(() {
                session = _initialize();
              }),
              child: const Text('Belum dapat membuka data. Coba lagi.'),
            ),
          ),
        );
      }
      return widget.store.user == null
          ? LoginScreen(store: widget.store)
          : MainScreen(store: widget.store);
    },
  );
}
```

Gambar 18. Kode langkah 1 bagian 4.

4.10.2 Langkah 2 Halaman Loading

Menampilkan indikator selama data awal aplikasi disiapkan.

File sumber: lib/gabungan_praktikum/loading_screen.dart

Bagian 1, baris 1 sampai 32.

Langkah 2 Bagian 1

```dart
import 'package:flutter/material.dart';

class LoadingScreen extends StatelessWidget {
  const LoadingScreen({super.key});
  @override
  Widget build(BuildContext context) => Scaffold(
    backgroundColor: const Color(0xFFFFBF00),
    body: SafeArea(
      child: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            ClipRRect(
              borderRadius: BorderRadius.circular(28),
              child: Image.asset(
                'lib/gabungan_praktikum/assets/expense_tracker_icon.png',
                width: 144,
                height: 144,
              ),
            ),
            const SizedBox(height: 20),
            const Text(
              'Expense Tracker',
              textAlign: TextAlign.center,
              style: TextStyle(
                fontSize: 28,
                fontWeight: FontWeight.bold,
                color: Color(0xFF172C43),
              ),
            ),
            const SizedBox(height: 28),
            const SizedBox(
```

Gambar 19. Kode langkah 2 bagian 1.

Bagian 2, baris 33 sampai 45.

Langkah 2 Bagian 2

```
            width: 24,
            height: 24,
            child: CircularProgressIndicator(
              strokeWidth: 2,
              color: Color(0xFF172C43),
            ),
          ),
        ],
      ),
    ),
  ),
);
}
```

Gambar 20. Kode langkah 2 bagian 2.

4.10.3 Langkah 3 Penyedia AppStore

Menyediakan objek penyimpanan agar dapat digunakan oleh widget dalam aplikasi.

File sumber: lib/gabungan_praktikum/store_scope.dart

Bagian 1, baris 1 sampai 29.

Langkah 3 Bagian 1

```dart
import 'package:flutter/material.dart';

import 'storage.dart';

class StoreScope extends InheritedWidget {
  final AppStore store;
  const StoreScope({super.key, required this.store, required super.child});
  static AppStore of(BuildContext context) =>
      context.dependOnInheritedWidgetOfExactType<StoreScope>()!.store;
  @override
  bool updateShouldNotify(StoreScope oldWidget) => oldWidget.store != store;
}

// Mempertahankan widget latihan asli; controller hanya dihubungkan ke draft akun.
mixin SavedInputs<T extends StatefulWidget> on State<T> {
  Map<String, TextEditingController> get savedControllers;
  String get draftKey;
  bool _loaded = false;
  bool _restoring = false;
  AppStore? _store;
  @override
  void didChangeDependencies() {
    super.didChangeDependencies();
    if (_loaded) return;
    _loaded = true;
    _store = StoreScope.of(context);
    _restore();
  }
```

Gambar 21. Kode langkah 3 bagian 1.

Bagian 2, baris 30 sampai 62.

Langkah 3 Bagian 2

```
  Future<void> _restore() async {
    try {
      final values = await _store!.draft(draftKey);
      if (!mounted) return;
      _restoring = true;
      for (final entry in savedControllers.entries) {
        if (entry.value.text.isEmpty) {
          entry.value.text = values[entry.key] ?? '';
        }
        entry.value.addListener(saveInputs);
      }
      _restoring = false;
      setState(() {});
    } catch (_) {
      if (mounted) _notifyFailure();
    }
  }

  void _notifyFailure() => ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(content: Text('Input belum tersimpan. Coba lagi.')),
  );
  void saveInputs() {
    if (_restoring || _store == null) return;
    _store!
        .saveDraft(
          draftKey,
          savedControllers.map((key, value) => MapEntry(key, value.text)),
        )
        .catchError((Object _) {
          if (mounted) _notifyFailure();
        });
  }
}
```

Gambar 22. Kode langkah 3 bagian 2.

4.10.4 Langkah 4 Model Transaksi

Mendefinisikan struktur transaksi, jenis transaksi, kategori, dan konversi data.

File sumber: lib/gabungan_praktikum/models/transaction_model.dart

Bagian 1, baris 1 sampai 32.

Langkah 4 Bagian 1

```dart
class TransactionModel {
  const TransactionModel({
    this.id,
    required this.userId,
    required this.title,
    required this.amount,
    required this.category,
    required this.date,
    required this.isIncome,
    this.isTransfer = false,
  });
  final int? id;
  final int userId;
  final String title;
  final int amount;
  final String category;
  final String date;
  final bool isIncome;
  final bool isTransfer;
  String get signedAmount =>
      '${isTransfer
          ? '↔'
          : isIncome
          ? '+'
          : '-'} ${rupiah(amount)}';
  Map<String, Object?> toMap() => {
    if (id != null) 'id': id,
    'user_id': userId,
    'title': title,
    'amount': amount,
    'category': category,
    'created_at': date,
```

Gambar 23. Kode langkah 4 bagian 1.

Bagian 2, baris 33 sampai 64.

Langkah 4 Bagian 2

```
    'kind': isTransfer
        ? 'transfer'
        : isIncome
        ? 'income'
        : 'expense',
  };
  factory TransactionModel.fromMap(Map<String, Object?> row) =>
      TransactionModel(
        id: row['id'] as int,
        userId: row['user_id'] as int,
        title: row['title'] as String,
        amount: row['amount'] as int,
        category: row['category'] as String,
        date: row['created_at'] as String,
        isIncome: row['kind'] == 'income',
        isTransfer: row['kind'] == 'transfer',
      );
}

const transactionCategories = [
  'Makanan',
  'Transportasi',
  'Belanja',
  'Hiburan',
  'Pemasukan',
  'Gaji',
  'Bonus',
  'Transfer',
  'Lainnya',
];
String displayDate(String value) {
  final date = DateTime.tryParse(value)?.toLocal();
```

Gambar 24. Kode langkah 4 bagian 2.

Bagian 3, baris 65 sampai 77.

Langkah 4 Bagian 3

```
  return date == null
      ? value
      : '${date.day.toString().padLeft(2, '0')}/${date.month.toString().padLeft(2, '0')}/${date.year}';
}

String rupiah(int value) {
  final digits = value.abs().toString();
  final formatted = digits.replaceAllMapped(
    RegExp(r'(\d)(?=(\d{3})+(?!\d))'),
    (m) => '${m[1]}.',
  );
  return '${value < 0 ? '-' : ''}Rp $formatted';
}
```

Gambar 25. Kode langkah 4 bagian 3.

4.10.5 Langkah 5 Penyimpanan SQLite dan Akun

Mengelola database, akun, verifikasi password, sesi, transaksi, saldo awal, dan draft.

File sumber: lib/gabungan_praktikum/storage.dart

Bagian 1, baris 1 sampai 30.

Langkah 5 Bagian 1

```dart
import 'dart:convert';
import 'dart:math';

import 'package:cryptography/cryptography.dart';
import 'package:flutter/foundation.dart';
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';

import 'models/transaction_model.dart';

class LocalUser {
  final int id;
  final String name, email;
  final int openingBalance;
  const LocalUser(this.id, this.name, this.email, this.openingBalance);
  factory LocalUser.fromMap(Map<String, Object?> row) => LocalUser(
    row['id'] as int,
    row['name'] as String,
    row['email'] as String,
    row['opening_balance'] as int,
  );
}

String? emailValidator(String? value) {
  if (value == null || value.trim().isEmpty) return 'Email wajib diisi.';
  return RegExp(r'^[^\s@]+@[^\s@]+\.[^\s@]+$').hasMatch(value.trim())
      ? null
      : 'Format email tidak valid.';
}
```

Gambar 26. Kode langkah 5 bagian 1.

Bagian 2, baris 31 sampai 62.

Langkah 5 Bagian 2

```
Future<String> _passwordKey(Map<String, String> input) async {
  final key = await Pbkdf2.hmacSha256(iterations: 600000, bits: 256).deriveKey(
    secretKey: SecretKey(utf8.encode(input['password']!)),
    nonce: base64Decode(input['salt']!),
  );
  return base64Encode(await key.extractBytes());
}

// Satu SQLite untuk akun, sesi, transaksi, dan input latihan. Tidak ada data contoh.
class AppStore {
  final DatabaseFactory? factory;
  final String? path;
  AppStore({this.factory, this.path});
  Future<Database>? _opening;
  LocalUser? user;
  int get userId => user?.id ?? (throw StateError('Silakan login kembali.'));
  Future<Database> get database => _opening ??= _open();
  Future<Database> _open() async {
    try {
      final driver = factory ?? databaseFactory;
      return await driver.openDatabase(
        path ??
            p.join(await driver.getDatabasesPath(), 'gabungan_praktikum.db'),
        options: OpenDatabaseOptions(
          version: 1,
          onConfigure: (db) => db.execute('PRAGMA foreign_keys = ON'),
          onCreate: (db, version) async {
            await db.execute(
              '''CREATE TABLE users(id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL, email TEXT NOT NULL UNIQUE, password_hash TEXT NOT NULL,
            password_salt TEXT NOT NULL, opening_balance INTEGER NOT NULL DEFAULT 0 CHECK(opening_balance
>= 0), created_at TEXT NOT NULL)''',
            );
```

Gambar 27. Kode langkah 5 bagian 2.

Bagian 3, baris 63 sampai 94.

Langkah 5 Bagian 3

```
            await db.execute(
              '''CREATE TABLE transactions(id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER NOT NULL REFERENCES users(id), title TEXT NOT NULL CHECK(length(trim(title))
> 0),
            amount INTEGER NOT NULL CHECK(amount > 0 AND amount <= 1000000000000), category TEXT NOT
NULL,
            kind TEXT NOT NULL CHECK(kind IN ('income','expense','transfer')), created_at TEXT NOT
NULL)''',
            );
            await db.execute(
              'CREATE INDEX tx_user_date ON transactions(user_id, created_at DESC)',
            );
            await db.execute(
              'CREATE TABLE session(id INTEGER PRIMARY KEY CHECK(id = 1), user_id INTEGER NOT NULL
REFERENCES users(id))',
            );
            await db.execute(
              'CREATE TABLE drafts(user_id INTEGER NOT NULL REFERENCES users(id), name TEXT NOT NULL,
payload TEXT NOT NULL, PRIMARY KEY(user_id,name))',
            );
          },
        ),
      );
    } catch (_) {
      _opening = null;
      rethrow;
    }
  }

  Future<void> restoreSession() async {
    final rows = await (await database).rawQuery(
      'SELECT users.* FROM users JOIN session ON session.user_id = users.id WHERE session.id = ?',
      [1],
    );
    user = rows.isEmpty ? null : LocalUser.fromMap(rows.first);
  }
```

Gambar 28. Kode langkah 5 bagian 3.

Bagian 4, baris 95 sampai 126.

Langkah 5 Bagian 4

```
Future<void> register(String name, String email, String password) async {
  if (name.trim().isEmpty ||
      emailValidator(email) != null ||
      password.length < 8) {
    throw const FormatException(
      'Periksa data pendaftaran. Password minimal 8 karakter.',
    );
  }
  final normalized = email.trim().toLowerCase();
  final db = await database;
  if ((await db.query(
    'users',
    columns: ['id'],
    where: 'email = ?',
    whereArgs: [normalized],
  )).isNotEmpty) {
    throw const FormatException('Email sudah terdaftar.');
  }
  final random = Random.secure();
  final salt = base64Encode(List.generate(16, (_) => random.nextInt(256)));
  final hash = await compute(_passwordKey, {
    'password': password,
    'salt': salt,
  });
  try {
    await db.insert('users', {
      'name': name.trim(),
      'email': normalized,
      'password_hash': hash,
      'password_salt': salt,
      'opening_balance': 0,
      'created_at': DateTime.now().toUtc().toIso8601String(),
```

Gambar 29. Kode langkah 5 bagian 4.

Bagian 5, baris 127 sampai 158.

Langkah 5 Bagian 5

```
      });
    } on DatabaseException catch (e) {
      if (e.isUniqueConstraintError()) {
        throw const FormatException('Email sudah terdaftar.');
      }
      rethrow;
    }
  }

  Future<bool> login(String email, String password) async {
    final db = await database;
    final rows = await db.query(
      'users',
      where: 'email = ?',
      whereArgs: [email.trim().toLowerCase()],
    );
    if (rows.isEmpty) return false;
    final row = rows.first;
    final actual = base64Decode(
      await compute(_passwordKey, {
        'password': password,
        'salt': row['password_salt'] as String,
      }),
    );
    final expected = base64Decode(row['password_hash'] as String);
    var difference = actual.length ^ expected.length;
    for (var i = 0; i < actual.length && i < expected.length; i++) {
      difference |= actual[i] ^ expected[i];
    }
    if (difference != 0) return false;
    await db.insert('session', {
      'id': 1,
```

Gambar 30. Kode langkah 5 bagian 5.

Bagian 6, baris 159 sampai 189.

Langkah 5 Bagian 6

```
      'user_id': row['id'],
    }, conflictAlgorithm: ConflictAlgorithm.replace);
    user = LocalUser.fromMap(row);
    return true;
  }

  Future<void> logout() async {
    await (await database).delete('session', where: 'id = ?', whereArgs: [1]);
    user = null;
  }

  Future<List<TransactionModel>> transactions() async {
    final owner = userId;
    return (await (await database).query(
      'transactions',
      where: 'user_id = ?',
      whereArgs: [owner],
      orderBy: 'created_at DESC, id DESC',
    )).map(TransactionModel.fromMap).toList();
  }

  Future<TransactionModel?> transaction(int id) async {
    final owner = userId;
    final rows = await (await database).query(
      'transactions',
      where: 'id = ? AND user_id = ?',
      whereArgs: [id, owner],
    );
    return rows.isEmpty ? null : TransactionModel.fromMap(rows.first);
  }
```

Gambar 31. Kode langkah 5 bagian 6.

Bagian 7, baris 190 sampai 219.

Langkah 5 Bagian 7

```
Future<void> saveTransaction(TransactionModel item) async {
  final owner = userId;
  if (item.userId != owner) {
    throw StateError('Transaksi tidak dapat diakses.');
  }
  if (item.title.trim().isEmpty ||
      item.amount <= 0 ||
      item.amount > 1000000000000 ||
      !transactionCategories.contains(item.category)) {
    throw const FormatException('Data transaksi tidak valid.');
  }
  final db = await database;
  if (item.id == null) {
    await db.insert('transactions', item.toMap());
  } else {
    final values = item.toMap()
      ..remove('id')
      ..remove('user_id');
    if (await db.update(
          'transactions',
          values,
          where: 'id = ? AND user_id = ?',
          whereArgs: [item.id, owner],
        ) !=
        1) {
      throw StateError('Transaksi tidak ditemukan.');
    }
  }
}
```

Gambar 32. Kode langkah 5 bagian 7.

Bagian 8, baris 220 sampai 245.

Langkah 5 Bagian 8

```dart
Future<void> deleteTransaction(int id) async {
  final owner = userId;
  if (await (await database).delete(
        'transactions',
        where: 'id = ? AND user_id = ?',
        whereArgs: [id, owner],
      ) !=
      1) {
    throw StateError('Transaksi tidak ditemukan.');
  }
}

Future<void> setOpeningBalance(int amount) async {
  final owner = userId;
  if (amount < 0 || amount > 1000000000000) {
    throw const FormatException('Saldo awal tidak valid.');
  }
  await (await database).update(
    'users',
    {'opening_balance': amount},
    where: 'id = ?',
    whereArgs: [owner],
  );
  user = LocalUser(owner, user!.name, user!.email, amount);
}
```

Gambar 33. Kode langkah 5 bagian 8.

Bagian 9, baris 246 sampai 277.

Langkah 5 Bagian 9

```
Future<void> updateProfile(String name, String email) async {
  final owner = userId;
  if (name.trim().isEmpty || emailValidator(email) != null) {
    throw const FormatException('Periksa nama dan email.');
  }
  final normalized = email.trim().toLowerCase();
  try {
    await (await database).update(
      'users',
      {'name': name.trim(), 'email': normalized},
      where: 'id = ?',
      whereArgs: [owner],
    );
    user = LocalUser(owner, name.trim(), normalized, user!.openingBalance);
  } on DatabaseException catch (error) {
    if (error.isUniqueConstraintError()) {
      throw const FormatException('Email sudah digunakan akun lain.');
    }
    rethrow;
  }
}

Future<Map<String, String>> draft(String name) async {
  final owner = userId;
  final rows = await (await database).query(
    'drafts',
    where: 'user_id = ? AND name = ?',
    whereArgs: [owner, name],
  );
  return rows.isEmpty
      ? {}
      : Map<String, String>.from(
```

Gambar 34. Kode langkah 5 bagian 9.

Bagian 10, baris 278 sampai 295.

Langkah 5 Bagian 10

```
            jsonDecode(rows.first['payload'] as String) as Map,
          );
  }

  Future<void> saveDraft(String name, Map<String, String> values) async {
    final owner = userId;
    await (await database).insert('drafts', {
      'user_id': owner,
      'name': name,
      'payload': jsonEncode(values),
    }, conflictAlgorithm: ConflictAlgorithm.replace);
  }

  Future<void> close() async {
    if (_opening != null) await (await _opening!).close();
    _opening = null;
  }
}
```

Gambar 35. Kode langkah 5 bagian 10.

4.10.6 Langkah 6 Halaman Login

Menyediakan form login, validasi, status proses, dan navigasi menuju halaman utama.

File sumber: lib/gabungan_praktikum/login_screen.dart

Bagian 1, baris 1 sampai 26.

Langkah 6 Bagian 1

```dart
import 'package:flutter/material.dart';

import 'storage.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key, required this.store});
  final AppStore store;
  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _formKey = GlobalKey<FormState>();
  final _email = TextEditingController();
  final _password = TextEditingController();
  bool _obscurePassword = true;
  bool _busy = false;
  String? _error;

  @override
  void dispose() {
    _email.dispose();
    _password.dispose();
    super.dispose();
  }
```

Gambar 36. Kode langkah 6 bagian 1.

Bagian 2, baris 27 sampai 57.

Langkah 6 Bagian 2

```
Future<void> _login() async {
  if (!_formKey.currentState!.validate()) return;
  setState(() {
    _busy = true;
    _error = null;
  });
  try {
    final valid = await widget.store.login(_email.text, _password.text);
    if (!mounted) return;
    if (valid) {
      Navigator.pushNamedAndRemoveUntil(context, '/main', (route) => false);
    } else {
      setState(() => _error = 'Email atau password salah.');
    }
  } catch (_) {
    if (mounted) setState(() => _error = 'Belum dapat masuk. Coba lagi.');
  } finally {
    if (mounted) setState(() => _busy = false);
  }
}

Future<void> _register() async {
  final result = await Navigator.pushNamed(context, '/register');
  if (!mounted || result is! String) return;
  _email.text = result;
  _password.clear();
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(content: Text('Akun berhasil dibuat. Silakan login.')),
  );
}
```

Gambar 37. Kode langkah 6 bagian 2.

Bagian 3, baris 58 sampai 89.

Langkah 6 Bagian 3

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Login'),
      automaticallyImplyLeading: false,
    ),
    body: Center(
      child: SingleChildScrollView(
        padding: const EdgeInsets.all(24),
        child: ConstrainedBox(
          constraints: const BoxConstraints(maxWidth: 440),
          child: Form(
            key: _formKey,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                const Icon(
                  Icons.account_balance_wallet,
                  size: 64,
                  color: Colors.amber,
                ),
                const SizedBox(height: 16),
                const Text(
                  'Expense Tracker',
                  textAlign: TextAlign.center,
                  style: TextStyle(fontSize: 26, fontWeight: FontWeight.bold),
                ),
                const SizedBox(height: 24),
                TextFormField(
                  controller: _email,
                  key: const Key('email'),
```

Gambar 38. Kode langkah 6 bagian 3.

Bagian 4, baris 90 sampai 121.

Langkah 6 Bagian 4

```
  keyboardType: TextInputType.emailAddress,
  decoration: const InputDecoration(
    labelText: 'Email',
    border: OutlineInputBorder(),
  ),
  validator: (value) {
    final email = value?.trim() ?? '';
    if (email.isEmpty) return 'Email wajib diisi.';
    if (!RegExp(r'^[^\s@]+@[^\s@]+\.[^\s@]+$')
        .hasMatch(email)) {
      return 'Format email tidak valid.';
    }
    return null;
  },
),
const SizedBox(height: 16),
TextFormField(
  controller: _password,
  key: const Key('password'),
  obscureText: _obscurePassword,
  decoration: InputDecoration(
    labelText: 'Password',
    border: const OutlineInputBorder(),
    suffixIcon: IconButton(
      tooltip: _obscurePassword
          ? 'Tampilkan password'
          : 'Sembunyikan password',
      onPressed: () => setState(
        () => _obscurePassword = !_obscurePassword,
      ),
      icon: Icon(
        _obscurePassword
```

Gambar 39. Kode langkah 6 bagian 4.

Bagian 5, baris 122 sampai 162.

Langkah 6 Bagian 5

```
                        ? Icons.visibility
                        : Icons.visibility_off,
                    ),
                  ),
                ),
                validator: (value) => value == null || value.trim().isEmpty
                    ? 'Password wajib diisi.'
                    : null,
                onFieldSubmitted: (_) => _login(),
              ),
              const SizedBox(height: 20),
              if (_error != null)
                Text(
                  _error!,
                  key: const Key('auth-error'),
                  style: const TextStyle(color: Colors.red),
                ),
              ElevatedButton(
                key: const Key('login-submit'),
                onPressed: _busy ? null : _login,
                child: Text(_busy ? 'Memproses...' : 'Login'),
              ),
              TextButton(
                key: const Key('open-register'),
                onPressed: _busy ? null : _register,
                child: const Text('Daftar akun baru'),
              ),
              const SizedBox(height: 12),
              const Text(
                'Masuk untuk melihat catatan keuanganmu.',
                textAlign: TextAlign.center,
              ),
            ],
          ),
        ),
      ),
    ),
  );
 }
}
```

Gambar 40. Kode langkah 6 bagian 5.

4.10.7 Langkah 7 Halaman Register

Menyediakan form pendaftaran beserta validasi dan penanganan hasil penyimpanan akun.

File sumber: lib/gabungan_praktikum/register_screen.dart

Bagian 1, baris 1 sampai 28.

Langkah 7 Bagian 1

```dart
import 'package:flutter/material.dart';

import 'storage.dart';

// Form registrasi melanjutkan pola Form/controller latihan Basic Form.
class RegisterScreen extends StatefulWidget {
  final AppStore store;
  const RegisterScreen({super.key, required this.store});
  @override
  State<RegisterScreen> createState() => _RegisterScreenState();
}

class _RegisterScreenState extends State<RegisterScreen> {
  final _form = GlobalKey<FormState>();
  final _name = TextEditingController(),
      _email = TextEditingController(),
      _password = TextEditingController(),
      _confirm = TextEditingController();
  bool _hidden = true, _busy = false;
  String? _error;
  @override
  void dispose() {
    for (final controller in [_name, _email, _password, _confirm]) {
      controller.dispose();
    }
    super.dispose();
  }
```

Gambar 41. Kode langkah 7 bagian 1.

Bagian 2, baris 29 sampai 60.

Langkah 7 Bagian 2

```
Future<void> _register() async {
  if (!_form.currentState!.validate()) return;
  setState(() {
    _busy = true;
    _error = null;
  });
  try {
    await widget.store.register(_name.text, _email.text, _password.text);
    if (mounted) Navigator.pop(context, _email.text.trim());
  } on FormatException catch (e) {
    if (mounted) setState(() => _error = e.message);
  } catch (error, stack) {
    debugPrint('Registrasi lokal gagal: $error\n$stack');
    if (mounted) setState(() => _error = 'Akun belum tersimpan. Coba lagi.');
  } finally {
    if (mounted) setState(() => _busy = false);
  }
}

@override
Widget build(BuildContext context) => Scaffold(
  appBar: AppBar(title: const Text('Daftar Akun')),
  body: SafeArea(
    child: Center(
      child: SingleChildScrollView(
        padding: const EdgeInsets.all(24),
        child: ConstrainedBox(
          constraints: const BoxConstraints(maxWidth: 440),
          child: Form(
            key: _form,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
```

Gambar 42. Kode langkah 7 bagian 2.

Bagian 3, baris 61 sampai 92.

Langkah 7 Bagian 3

```
children: [
  const Icon(
    Icons.account_balance_wallet,
    size: 64,
    color: Colors.amber,
  ),
  const SizedBox(height: 24),
  TextFormField(
    key: const Key('register-name'),
    controller: _name,
    decoration: const InputDecoration(
      labelText: 'Nama lengkap',
      border: OutlineInputBorder(),
    ),
    validator: (v) => v == null || v.trim().isEmpty
        ? 'Nama wajib diisi.'
        : null,
  ),
  const SizedBox(height: 16),
  TextFormField(
    key: const Key('register-email'),
    controller: _email,
    keyboardType: TextInputType.emailAddress,
    decoration: const InputDecoration(
      labelText: 'Email',
      border: OutlineInputBorder(),
    ),
    validator: emailValidator,
  ),
  const SizedBox(height: 16),
  TextFormField(
    key: const Key('register-password'),
```

Gambar 43. Kode langkah 7 bagian 3.

Bagian 4, baris 93 sampai 124.

Langkah 7 Bagian 4

```
  controller: _password,
  obscureText: _hidden,
  decoration: InputDecoration(
    labelText: 'Password',
    border: const OutlineInputBorder(),
    suffixIcon: IconButton(
      tooltip: 'Tampilkan/sembunyikan password',
      onPressed: () => setState(() => _hidden = !_hidden),
      icon: Icon(
        _hidden ? Icons.visibility : Icons.visibility_off,
      ),
    ),
  ),
  validator: (v) => (v?.length ?? 0) < 8
      ? 'Password minimal 8 karakter.'
      : null,
),
const SizedBox(height: 16),
TextFormField(
  key: const Key('register-confirm'),
  controller: _confirm,
  obscureText: _hidden,
  decoration: const InputDecoration(
    labelText: 'Konfirmasi password',
    border: OutlineInputBorder(),
  ),
  validator: (v) =>
      v == null || v.isEmpty || v != _password.text
      ? 'Konfirmasi password tidak sama.'
      : null,
),
if (_error != null)
```

Gambar 44. Kode langkah 7 bagian 4.

Bagian 5, baris 125 sampai 147.

Langkah 7 Bagian 5

```
                  Padding(
                    padding: const EdgeInsets.all(8),
                    child: Text(
                      _error!,
                      key: const Key('register-error'),
                      style: const TextStyle(color: Colors.red),
                    ),
                  ),
                const SizedBox(height: 24),
                ElevatedButton(
                  key: const Key('register-submit'),
                  onPressed: _busy ? null : _register,
                  child: Text(_busy ? 'Memproses...' : 'Daftar'),
                ),
              ],
            ),
          ),
        ),
      ),
    ),
  );
}
```

Gambar 45. Kode langkah 7 bagian 5.

4.10.8 Langkah 8 Navigasi Utama

Menghubungkan tab Beranda, Riwayat, dan Profil melalui navigasi bawah.

File sumber: lib/gabungan_praktikum/main_screen.dart

Bagian 1, baris 1 sampai 28.

Langkah 8 Bagian 1

```dart
import 'package:flutter/material.dart';

import 'dashboard_screen.dart';
import 'history_screen.dart';
import 'profile_screen.dart';
import 'models/transaction_model.dart';
import 'storage.dart';
import 'loading_screen.dart';

// Kerangka tab dan IndexedStack dipertahankan dari pertemuan_05.
class MainScreen extends StatefulWidget {
  const MainScreen({super.key, required this.store});
  final AppStore store;
  @override
  State<MainScreen> createState() => _MainScreenState();
}

class _MainScreenState extends State<MainScreen> {
  int _selectedIndex = 0;
  List<TransactionModel> _transactions = [];
  bool _loading = true;
  String? _error;
  @override
  void initState() {
    super.initState();
    _refresh();
  }
```

Gambar 46. Kode langkah 8 bagian 1.

Bagian 2, baris 29 sampai 60.

Langkah 8 Bagian 2

```
Future<void> _refresh() async {
  try {
    final items = await widget.store.transactions();
    if (mounted) {
      setState(() {
        _transactions = items;
        _loading = false;
        _error = null;
      });
    }
  } catch (_) {
    if (mounted) {
      setState(() {
        _error = 'Transaksi belum dapat dibuka.';
        _loading = false;
      });
    }
  }
}

void _onItemTapped(int index) => setState(() => _selectedIndex = index);
@override
Widget build(BuildContext context) {
  if (widget.store.user == null) return const SizedBox.shrink();
  if (_loading) return const LoadingScreen();
  return Scaffold(
    body: _error != null
        ? Center(
            child: TextButton(
              onPressed: _refresh,
              child: Text('$_error Coba lagi.'),
            ),
```

Gambar 47. Kode langkah 8 bagian 2.

Bagian 3, baris 61 sampai 93.

Langkah 8 Bagian 3

```
          )
        : IndexedStack(
            index: _selectedIndex,
            children: [
              DashboardScreen(
                user: widget.store.user!,
                store: widget.store,
                transactions: _transactions,
                onAdded: (_) => _refresh(),
                onHistory: () => _onItemTapped(1),
                onProfile: () => _onItemTapped(2),
              ),
              HistoryScreen(
                transactions: _transactions,
                store: widget.store,
                onChanged: _refresh,
              ),
              ProfileScreen(store: widget.store, onChanged: _refresh),
            ],
          ),
      bottomNavigationBar: BottomNavigationBar(
        currentIndex: _selectedIndex,
        onTap: _onItemTapped,
        selectedItemColor: Colors.amber[800],
        items: const [
          BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Beranda'),
          BottomNavigationBarItem(icon: Icon(Icons.history), label: 'Riwayat'),
          BottomNavigationBarItem(icon: Icon(Icons.person), label: 'Profil'),
        ],
      ),
    );
  }
}
```

Gambar 48. Kode langkah 8 bagian 3.

4.10.9 Langkah 9 Dashboard

Menyusun sapaan pengguna, ringkasan saldo, tombol aksi, dan daftar transaksi.

File sumber: lib/gabungan_praktikum/dashboard_screen.dart

Bagian 1, baris 1 sampai 30.

Langkah 9 Bagian 1

```dart
import 'package:flutter/material.dart';

import 'models/transaction_model.dart';
import 'widgets/transaction_tile.dart';
import 'storage.dart';
import 'widgets/action_buttons_widget.dart';
import 'widgets/balance_card_widget.dart';

class DashboardScreen extends StatelessWidget {
  const DashboardScreen({
    super.key,
    required this.user,
    required this.store,
    required this.transactions,
    required this.onAdded,
    required this.onHistory,
    required this.onProfile,
  });
  final LocalUser user;
  final AppStore store;
  final List<TransactionModel> transactions;
  final ValueChanged<TransactionModel> onAdded;
  final VoidCallback onHistory;
  final VoidCallback onProfile;

  Future<void> _openAdd(BuildContext context, [String kind = 'expense']) async {
    final result = await Navigator.pushNamed(context, '/add', arguments: kind);
    if (result is TransactionModel) onAdded(result);
  }
```

Gambar 49. Kode langkah 9 bagian 1.

Bagian 2, baris 31 sampai 62.

Langkah 9 Bagian 2

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Dashboard Keuangan'),
      backgroundColor: Colors.amber,
      foregroundColor: Colors.white,
    ),
    floatingActionButton: FloatingActionButton(
      backgroundColor: Colors.amber,
      foregroundColor: Colors.white,
      tooltip: 'Tambah transaksi',
      onPressed: () => _openAdd(context),
      child: const Icon(Icons.add),
    ),
    body: SingleChildScrollView(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            'Halo, ${user.name}',
            style: const TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
          ),
          const SizedBox(height: 20),
          Text(user.email),
          const SizedBox(height: 12),
          BalanceCardWidget(
            balance: transactions.fold<int>(
              user.openingBalance,
              (sum, item) =>
                  sum +
```

Gambar 50. Kode langkah 9 bagian 2.

Bagian 3, baris 63 sampai 94.

Langkah 9 Bagian 3

```
            (item.isTransfer
                ? 0
                : item.isIncome
                ? item.amount
                : -item.amount),
      ),
    ),
    const SizedBox(height: 12),
    ActionButtonsWidget(onAction: (kind) => _openAdd(context, kind)),
    Row(
      mainAxisAlignment: MainAxisAlignment.spaceEvenly,
      children: [
        IconButton(
          tooltip: 'Kalkulator Kabataku',
          onPressed: () => Navigator.pushNamed(context, '/calculator'),
          icon: const Icon(Icons.calculate),
        ),
        IconButton(
          tooltip: 'Catat transaksi',
          onPressed: () => _openAdd(context),
          icon: const Icon(Icons.add_circle_outline),
        ),
        IconButton(
          tooltip: 'Riwayat',
          onPressed: onHistory,
          icon: const Icon(Icons.history),
        ),
        IconButton(
          tooltip: 'Profil',
          onPressed: onProfile,
          icon: const Icon(Icons.person),
        ),
```

Gambar 51. Kode langkah 9 bagian 3.

Bagian 4, baris 95 sampai 134.

Langkah 9 Bagian 4

```
            ],
          ),
          const SizedBox(height: 20),
          const Text(
            'Transaksi Terakhir',
            style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
          ),
          const SizedBox(height: 12),
          _buildTransactionList(context),
          const SizedBox(height: 80),
        ],
      ),
    ),
  );
}

Widget _buildTransactionList(BuildContext context) {
  if (transactions.isEmpty) {
    return const Padding(
      padding: EdgeInsets.all(16),
      child: Text('Belum ada transaksi.'),
    );
  }
  return Card(
    elevation: 2,
    child: Column(
      children: [
        for (int i = 0; i < transactions.length; i++) ...[
          TransactionTile(
            transaction: transactions[i],
            store: store,
            onChanged: () => onAdded(transactions[i]),
          ),
          if (i < transactions.length - 1) const Divider(height: 1),
        ],
      ],
    ),
  );
}
}
```

Gambar 52. Kode langkah 9 bagian 4.

4.10.10 Langkah 10 Kartu Saldo

Menampilkan saldo beserta kontrol untuk menyembunyikan atau memperlihatkannya.

File sumber: lib/gabungan_praktikum/widgets/balance_card_widget.dart

Bagian 1, baris 1 sampai 24.

Langkah 10 Bagian 1

```dart
import 'package:flutter/material.dart';

import '../models/transaction_model.dart';

// State dan toggle diadaptasi dari modul widgets; warna mengikuti kelanjutan Modul V.
class BalanceCardWidget extends StatefulWidget {
  const BalanceCardWidget({super.key, required this.balance});
  final int balance;

  @override
  State<BalanceCardWidget> createState() => _BalanceCardWidgetState();
}

class _BalanceCardWidgetState extends State<BalanceCardWidget> {
  // Menyimpan status apakah saldo ditampilkan atau disembunyikan
  bool _isBalanceVisible = true;

  // Mengubah status tampilan saldo
  void _toggleVisibility() {
    setState(() {
      _isBalanceVisible = !_isBalanceVisible;
    });
  }
```

Gambar 53. Kode langkah 10 bagian 1.

Bagian 2, baris 25 sampai 56.

Langkah 10 Bagian 2

```
@override
Widget build(BuildContext context) {
  return Card(
    elevation: 4,
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
    color: Colors.amber.shade700,
    child: Padding(
      padding: const EdgeInsets.all(20.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              const Expanded(
                child: Text(
                  'Saldo Utama',
                  style: TextStyle(fontSize: 16, color: Colors.white70),
                ),
              ),
              IconButton(
                tooltip: _isBalanceVisible
                    ? 'Sembunyikan saldo'
                    : 'Tampilkan saldo',
                icon: Icon(
                  _isBalanceVisible ? Icons.visibility : Icons.visibility_off,
                  color: Colors.white,
                ),
                onPressed: _toggleVisibility,
              ),
            ],
          ),
```

Gambar 54. Kode langkah 10 bagian 2.

Bagian 3, baris 57 sampai 71.

Langkah 10 Bagian 3

```
            const SizedBox(height: 8),
            Text(
              _isBalanceVisible ? rupiah(widget.balance) : 'Rp *********',
              style: const TextStyle(
                fontSize: 28,
                fontWeight: FontWeight.bold,
                color: Colors.white,
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

Gambar 55. Kode langkah 10 bagian 3.

4.10.11 Langkah 11 Tombol Aksi

Menyusun tombol aksi pencatatan dan fitur pendukung pada halaman utama.

File sumber: lib/gabungan_praktikum/widgets/action_buttons_widget.dart

Bagian 1, baris 1 sampai 32.

Langkah 11 Bagian 1

```dart
import 'package:flutter/material.dart';

// Disalin dari modul layout, kini tiap tombol memiliki callback.
class ActionButtonsWidget extends StatelessWidget {
  const ActionButtonsWidget({super.key, required this.onAction});
  final ValueChanged<String> onAction;

  @override
  Widget build(BuildContext context) {
    return Row(
      mainAxisAlignment: MainAxisAlignment.spaceEvenly,
      children: [
        _buildActionButton(Icons.arrow_downward, 'Pemasukan', Colors.green),
        _buildActionButton(Icons.arrow_upward, 'Pengeluaran', Colors.red),
        _buildActionButton(Icons.swap_horiz, 'Transfer', Colors.blue),
      ],
    );
  }

  Widget _buildActionButton(IconData icon, String label, Color color) {
    return Expanded(
      child: InkWell(
        onTap: () => onAction(
          label == 'Pemasukan'
              ? 'income'
              : label == 'Pengeluaran'
              ? 'expense'
              : 'transfer',
        ),
        child: Column(
          children: [
            Container(
```

Gambar 56. Kode langkah 11 bagian 1.

Bagian 2, baris 33 sampai 51.

Langkah 11 Bagian 2

```
            padding: const EdgeInsets.all(12),
            decoration: BoxDecoration(
              color: color.withValues(alpha: 0.2),
              borderRadius: BorderRadius.circular(12),
            ),
            child: Icon(icon, color: color, size: 28),
          ),
          const SizedBox(height: 8),
          Text(
            label,
            textAlign: TextAlign.center,
            style: const TextStyle(fontWeight: FontWeight.w500),
          ),
        ],
      ),
    ),
  );
 }
}
```

Gambar 57. Kode langkah 11 bagian 2.

4.10.12 Langkah 12 Riwayat Transaksi

Menampilkan daftar transaksi yang dimiliki akun aktif.

File sumber: lib/gabungan_praktikum/history_screen.dart

Bagian 1, baris 1 sampai 37.

Langkah 12 Bagian 1

```dart
import 'package:flutter/material.dart';

import 'models/transaction_model.dart';
import 'widgets/transaction_tile.dart';
import 'storage.dart';

class HistoryScreen extends StatelessWidget {
  const HistoryScreen({
    super.key,
    required this.transactions,
    required this.store,
    required this.onChanged,
  });
  final AppStore store;
  final VoidCallback onChanged;
  final List<TransactionModel> transactions;
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Riwayat Transaksi'),
        automaticallyImplyLeading: false,
      ),
      body: transactions.isEmpty
          ? const Center(child: Text('Belum ada transaksi.'))
          : ListView.separated(
              itemCount: transactions.length,
              separatorBuilder: (_, index) => const Divider(height: 1),
              itemBuilder: (_, index) => TransactionTile(
                transaction: transactions[index],
                store: store,
                onChanged: onChanged,
              ),
            ),
    );
  }
}
```

Gambar 58. Kode langkah 12 bagian 1.

4.10.13 Langkah 13 Item Transaksi

Menampilkan ringkasan satu transaksi serta membuka halaman detailnya.

File sumber: lib/gabungan_praktikum/widgets/transaction_tile.dart

Bagian 1, baris 1 sampai 32.

Langkah 13 Bagian 1

```dart
import 'package:flutter/material.dart';

import '../models/transaction_model.dart';
import '../transaction_detail_screen.dart';
import '../storage.dart';

class TransactionTile extends StatelessWidget {
  const TransactionTile({
    super.key,
    required this.transaction,
    required this.store,
    required this.onChanged,
  });
  final AppStore store;
  final VoidCallback onChanged;
  final TransactionModel transaction;
  @override
  Widget build(BuildContext context) {
    final amount = transaction.signedAmount;
    return LayoutBuilder(
      builder: (context, constraints) {
        final compact =
            constraints.maxWidth < 350 ||
            MediaQuery.textScalerOf(context).scale(14) > 20 ||
            transaction.amount >= 1000000000;
        return ListTile(
          leading: CircleAvatar(
            backgroundColor: transaction.isTransfer
                ? Colors.blue
                : transaction.isIncome
                ? Colors.green
                : Colors.orange,
```

Gambar 59. Kode langkah 13 bagian 1.

Bagian 2, baris 33 sampai 75.

Langkah 13 Bagian 2

```
            child: Icon(
              transaction.isTransfer
                  ? Icons.swap_horiz
                  : transaction.isIncome
                  ? Icons.attach_money
                  : Icons.fastfood,
              color: Colors.white,
            ),
          ),
          title: Text(transaction.title),
          subtitle: Text(
            '${displayDate(transaction.date)} • ${transaction.category}${compact ? '\n$amount' : ''}',
          ),
          trailing: compact
              ? null
              : Text(
                  amount,
                  style: TextStyle(
                    color: transaction.isTransfer
                        ? Colors.blue
                        : transaction.isIncome
                        ? Colors.green
                        : Colors.red,
                    fontWeight: FontWeight.bold,
                  ),
                ),
          onTap: () async {
            await Navigator.push(
              context,
              MaterialPageRoute<void>(
                builder: (_) => TransactionDetailScreen(
                  transaction: transaction,
                  store: store,
                ),
              ),
            );
            onChanged();
          },
        );
      },
    );
  }
}
```

Gambar 60. Kode langkah 13 bagian 2.

4.10.14 Langkah 14 Profil dan Logout

Menampilkan identitas akun, pengaturan saldo awal, dan fungsi logout.

File sumber: lib/gabungan_praktikum/profile_screen.dart

Bagian 1, baris 1 sampai 28.

Langkah 14 Bagian 1

```dart
import 'package:flutter/material.dart';

import 'models/transaction_model.dart';
import 'storage.dart';

class ProfileScreen extends StatelessWidget {
  const ProfileScreen({
    super.key,
    required this.store,
    required this.onChanged,
  });
  final AppStore store;
  final VoidCallback onChanged;
  Future<void> _logout(BuildContext context) async {
    try {
      await store.logout();
      if (context.mounted) {
        Navigator.pushNamedAndRemoveUntil(context, '/login', (route) => false);
      }
    } catch (_) {
      if (context.mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('Belum dapat logout. Coba lagi.')),
        );
      }
    }
  }
```

Gambar 61. Kode langkah 14 bagian 1.

Bagian 2, baris 29 sampai 60.

Langkah 14 Bagian 2

```dart
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Profil'),
      automaticallyImplyLeading: false,
    ),
    body: ListView(
      padding: const EdgeInsets.all(24),
      children: [
        const Center(
          child: CircleAvatar(
            radius: 45,
            backgroundColor: Colors.amber,
            child: Icon(Icons.person, size: 56, color: Colors.white),
          ),
        ),
        const SizedBox(height: 20),
        Text(
          store.user!.name,
          textAlign: TextAlign.center,
          style: const TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
        ),
        const SizedBox(height: 8),
        Text(store.user!.email, textAlign: TextAlign.center),
        const SizedBox(height: 28),
        ListTile(
          leading: const Icon(Icons.edit),
          title: const Text('Edit profil'),
          onTap: () async {
            await Navigator.pushNamed(context, '/edit-profile');
            if (context.mounted) onChanged();
```

Gambar 62. Kode langkah 14 bagian 2.

Bagian 3, baris 61 sampai 88.

Langkah 14 Bagian 3

```
        },
      ),
      ListTile(
        title: const Text('Saldo awal'),
        subtitle: Text(rupiah(store.user!.openingBalance)),
        leading: const Icon(Icons.account_balance_wallet),
        onTap: () async {
          final amount = await showDialog<int>(
            context: context,
            builder: (_) =>
                _OpeningBalanceDialog(amount: store.user!.openingBalance),
          );
          if (!context.mounted || amount == null) return;
          try {
            await store.setOpeningBalance(amount);
            onChanged();
          } catch (_) {
            if (context.mounted) {
              ScaffoldMessenger.of(context).showSnackBar(
                const SnackBar(
                  content: Text('Saldo awal belum tersimpan.'),
                ),
              );
            }
          }
        },
      ),
```

Gambar 63. Kode langkah 14 bagian 3.

Bagian 4, baris 89 sampai 117.

Langkah 14 Bagian 4

```
          const SizedBox(height: 20),
          ElevatedButton.icon(
            key: const Key('logout'),
            onPressed: () => _logout(context),
            icon: const Icon(Icons.logout),
            label: const Text('Logout'),
          ),
        ],
      ),
    );
  }
}

class _OpeningBalanceDialog extends StatefulWidget {
  final int amount;
  const _OpeningBalanceDialog({required this.amount});
  @override
  State<_OpeningBalanceDialog> createState() => _OpeningBalanceDialogState();
}

class _OpeningBalanceDialogState extends State<_OpeningBalanceDialog> {
  final _form = GlobalKey<FormState>();
  late final _amount = TextEditingController(text: '${widget.amount}');
  @override
  void dispose() {
    _amount.dispose();
    super.dispose();
  }
```

Gambar 64. Kode langkah 14 bagian 4.

Bagian 5, baris 118 sampai 151.

Langkah 14 Bagian 5

```
  @override
  Widget build(BuildContext context) => AlertDialog(
    title: const Text('Saldo awal'),
    content: Form(
      key: _form,
      child: TextFormField(
        key: const Key('opening-balance'),
        controller: _amount,
        keyboardType: TextInputType.number,
        decoration: const InputDecoration(labelText: 'Nominal (Rp)'),
        validator: (value) {
          final amount = int.tryParse(value ?? '');
          return amount == null || amount < 0 || amount > 1000000000000
              ? 'Masukkan angka 0 sampai 1000000000000.'
              : null;
        },
      ),
    ),
    actions: [
      TextButton(
        onPressed: () => Navigator.pop(context),
        child: const Text('Batal'),
      ),
      TextButton(
        onPressed: () {
          if (_form.currentState!.validate()) {
            Navigator.pop(context, int.parse(_amount.text));
          }
        },
        child: const Text('Simpan'),
      ),
    ],
  );
}
```

Gambar 65. Kode langkah 14 bagian 5.

4.10.15 Langkah 15 Edit Profil

Menyediakan form untuk memperbarui nama dan email pengguna.

File sumber: lib/gabungan_praktikum/edit_profile_screen.dart

Bagian 1, baris 1 sampai 30.

Langkah 15 Bagian 1

```dart
import 'package:flutter/material.dart';

import 'storage.dart';

// Form, controller, validator, dan padding dari pertemuan_01/textformfield.dart.
// Isian nama/email dipakai sebagai edit akun, bukan menu latihan terpisah.
class EditProfileScreen extends StatefulWidget {
  final AppStore store;
  const EditProfileScreen({super.key, required this.store});
  @override
  State<EditProfileScreen> createState() => _EditProfileScreenState();
}

class _EditProfileScreenState extends State<EditProfileScreen> {
  final _formKey = GlobalKey<FormState>();
  late final _nameController = TextEditingController(
    text: widget.store.user!.name,
  );
  late final _emailController = TextEditingController(
    text: widget.store.user!.email,
  );
  bool _saving = false;
  String? _error;
  @override
  void dispose() {
    _nameController.dispose();
    _emailController.dispose();
    super.dispose();
  }
```

Gambar 66. Kode langkah 15 bagian 1.

Bagian 2, baris 31 sampai 57.

Langkah 15 Bagian 2

```
Future<void> _submitForm() async {
  if (!_formKey.currentState!.validate()) return;
  setState(() {
    _saving = true;
    _error = null;
  });
  try {
    await widget.store.updateProfile(
      _nameController.text,
      _emailController.text,
    );
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Profil berhasil disimpan.')),
    );
    Navigator.pop(context, true);
  } on FormatException catch (error) {
    if (mounted) setState(() => _error = error.message);
  } catch (_) {
    if (mounted) {
      setState(() => _error = 'Profil belum tersimpan. Coba lagi.');
    }
  } finally {
    if (mounted) setState(() => _saving = false);
  }
}
```

Gambar 67. Kode langkah 15 bagian 2.

Bagian 3, baris 58 sampai 89.

Langkah 15 Bagian 3

```
@override
Widget build(BuildContext context) => Scaffold(
  appBar: AppBar(title: const Text('Edit Profil')),
  body: SafeArea(
    child: SingleChildScrollView(
      child: Form(
        key: _formKey,
        child: Padding(
          padding: const EdgeInsets.all(20),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              const SizedBox(height: 10),
              TextFormField(
                key: const Key('profile-name'),
                controller: _nameController,
                decoration: const InputDecoration(
                  labelText: 'Nama lengkap',
                  border: OutlineInputBorder(),
                ),
                validator: (value) => value == null || value.trim().isEmpty
                    ? 'Nama wajib diisi.'
                    : null,
              ),
              const SizedBox(height: 10),
              TextFormField(
                key: const Key('profile-email'),
                controller: _emailController,
                keyboardType: TextInputType.emailAddress,
                decoration: const InputDecoration(
                  labelText: 'Email',
                  border: OutlineInputBorder(),
```

Gambar 68. Kode langkah 15 bagian 3.

Bagian 4, baris 90 sampai 108.

Langkah 15 Bagian 4

```
                ),
                validator: emailValidator,
              ),
              const SizedBox(height: 10),
              if (_error != null)
                Text(_error!, style: const TextStyle(color: Colors.red)),
              ElevatedButton(
                key: const Key('profile-save'),
                onPressed: _saving ? null : _submitForm,
                child: Text(_saving ? 'Menyimpan...' : 'Simpan'),
              ),
            ],
          ),
        ),
      ),
    ),
  );
}
```

Gambar 69. Kode langkah 15 bagian 4.

4.10.16 Langkah 16 Form Transaksi

Menangani input transaksi, validasi, tanggal, kategori, draft, dan penyimpanan.

File sumber: lib/gabungan_praktikum/add_transaction_screen.dart

Bagian 1, baris 1 sampai 31.

Langkah 16 Bagian 1

```dart
import 'package:flutter/material.dart';

import 'models/transaction_model.dart';
import 'storage.dart';

class AddTransactionScreen extends StatefulWidget {
  const AddTransactionScreen({
    super.key,
    required this.store,
    this.initialKind = 'expense',
    this.transaction,
  });
  final AppStore store;
  final String initialKind;
  final TransactionModel? transaction;

  @override
  State<AddTransactionScreen> createState() => _AddTransactionScreenState();
}

class _AddTransactionScreenState extends State<AddTransactionScreen> {
  String _category = 'Makanan';
  bool _isIncome = false;
  bool _isTransfer = false;
  bool _saving = false;
  final _formKey = GlobalKey<FormState>();

  final _titleController = TextEditingController();
  final _amountController = TextEditingController();
  final _dateController = TextEditingController();
```

Gambar 70. Kode langkah 16 bagian 1.

Bagian 2, baris 32 sampai 61.

Langkah 16 Bagian 2

```
@override
void initState() {
  super.initState();
  _isIncome = widget.initialKind == 'income';
  _isTransfer = widget.initialKind == 'transfer';
  _category = _isIncome
      ? 'Pemasukan'
      : _isTransfer
      ? 'Transfer'
      : 'Makanan';
  final item = widget.transaction;
  if (item != null) {
    _titleController.text = item.title;
    _amountController.text = '${item.amount}';
    _dateController.text = displayDate(item.date);
    _category = item.category;
    _isIncome = item.isIncome;
    _isTransfer = item.isTransfer;
  } else {
    _loadDraft();
  }
  for (final controller in [
    _titleController,
    _amountController,
    _dateController,
  ]) {
    controller.addListener(_saveDraft);
  }
}
```

Gambar 71. Kode langkah 16 bagian 2.

Bagian 3, baris 62 sampai 93.

Langkah 16 Bagian 3

```
bool _restoring = false;
Future<void> _loadDraft() async {
  try {
    final draft = await widget.store.draft('transaction');
    if (!mounted || draft.isEmpty || _titleController.text.isNotEmpty) return;
    _restoring = true;
    setState(() {
      _titleController.text = draft['title'] ?? '';
      _amountController.text = draft['amount'] ?? '';
      _dateController.text = draft['date'] ?? '';
      _category = draft['category'] ?? _category;
      _isIncome = draft['income'] == 'true';
      _isTransfer = draft['transfer'] == 'true';
    });
    _restoring = false;
  } catch (_) {
    if (mounted) _notice('Input belum dapat dibuka.');
  }
}

void _notice(String text) =>
    ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(text)));
void _saveDraft() {
  if (widget.transaction != null || _restoring || _saving) return;
  widget.store
      .saveDraft('transaction', {
        'title': _titleController.text,
        'amount': _amountController.text,
        'date': _dateController.text,
        'category': _category,
        'income': '$_isIncome',
        'transfer': '$_isTransfer',
```

Gambar 72. Kode langkah 16 bagian 3.

Bagian 4, baris 94 sampai 122.

Langkah 16 Bagian 4

```
        })
        .catchError((Object _) {
          if (mounted) _notice('Input belum tersimpan. Coba lagi.');
        });
  }

  @override
  void dispose() {
    _titleController.dispose();
    _amountController.dispose();
    _dateController.dispose();
    super.dispose();
  }

  Future<void> _selectDate() async {
    final DateTime? selectedDate = await showDatePicker(
      context: context,
      initialDate: DateTime.now(),
      firstDate: DateTime(2000),
      lastDate: DateTime(2100),
      helpText: 'Pilih Tanggal Transaksi',
      cancelText: 'Batal',
      confirmText: 'Pilih',
    );

    if (!mounted || selectedDate == null) {
      return;
    }
```

Gambar 73. Kode langkah 16 bagian 4.

Bagian 5, baris 123 sampai 154.

Langkah 16 Bagian 5

```
    final String day = selectedDate.day.toString().padLeft(2, '0');
    final String month = selectedDate.month.toString().padLeft(2, '0');
    final String year = selectedDate.year.toString();

    setState(() {
      _dateController.text = '$day/$month/$year';
    });
  }

  Future<void> _saveTransaction() async {
    if (_formKey.currentState!.validate()) {
      setState(() => _saving = true);
      final parts = _dateController.text.split('/').map(int.parse).toList();
      final now = DateTime.now();
      final item = TransactionModel(
        id: widget.transaction?.id,
        userId: widget.store.userId,
        title: _titleController.text.trim(),
        amount: int.parse(_amountController.text.trim()),
        category: _category,
        date: DateTime(
          parts[2],
          parts[1],
          parts[0],
          now.hour,
          now.minute,
          now.second,
        ).toUtc().toIso8601String(),
        isIncome: _isIncome,
        isTransfer: _isTransfer,
      );
      try {
```

Gambar 74. Kode langkah 16 bagian 5.

Bagian 6, baris 155 sampai 178.

Langkah 16 Bagian 6

```
        await widget.store.saveTransaction(item);
        if (widget.transaction == null) {
          await widget.store.saveDraft('transaction', {});
        }
        if (!mounted) return;
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(
            content: Text(
              'Transaksi ${_titleController.text} '
              'pada ${_dateController.text} berhasil disimpan!',
            ),
            backgroundColor: Colors.green,
          ),
        );

        Navigator.pop(context, item);
      } catch (_) {
        if (mounted) _notice('Transaksi belum tersimpan. Coba lagi.');
      } finally {
        if (mounted) setState(() => _saving = false);
      }
    }
  }
```

Gambar 75. Kode langkah 16 bagian 6.

Bagian 7, baris 179 sampai 210.

Langkah 16 Bagian 7

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      actions: [
        IconButton(
          tooltip: 'Kalkulator Kabataku',
          onPressed: () => Navigator.pushNamed(context, '/calculator'),
          icon: const Icon(Icons.calculate),
        ),
      ],
      title: Text(
        widget.transaction == null ? 'Catat Transaksi' : 'Edit Transaksi',
      ),
      backgroundColor: Colors.amber,
      foregroundColor: Colors.white,
    ),
    body: SingleChildScrollView(
      padding: const EdgeInsets.all(16),
      child: Form(
        key: _formKey,
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            TextFormField(
              key: const Key('tx-title'),
              controller: _titleController,
              decoration: const InputDecoration(
                labelText: 'Judul',
                hintText: 'Contoh: Belanja Bulanan',
                prefixIcon: Icon(Icons.title),
                border: OutlineInputBorder(),
```

Gambar 76. Kode langkah 16 bagian 7.

Bagian 8, baris 211 sampai 241.

Langkah 16 Bagian 8

```
    ),
    validator: (value) {
      if (value == null || value.trim().isEmpty) {
        return 'Judul tidak boleh kosong';
      }

      return null;
    },
  ),
  const SizedBox(height: 16),
  TextFormField(
    key: const Key('tx-amount'),
    controller: _amountController,
    keyboardType: TextInputType.number,
    decoration: const InputDecoration(
      labelText: 'Nominal',
      hintText: 'Contoh: 100000',
      prefixIcon: Icon(Icons.attach_money),
      border: OutlineInputBorder(),
    ),
    validator: (value) {
      if (value == null || value.trim().isEmpty) {
        return 'Nominal tidak boleh kosong';
      }

      final int? amount = int.tryParse(value.trim());

      if (amount == null) {
        return 'Nominal harus berupa angka';
      }
```

Gambar 77. Kode langkah 16 bagian 8.

Bagian 9, baris 242 sampai 268.

Langkah 16 Bagian 9

```
      if (amount <= 0 || amount > 1000000000000) {
        return 'Nominal harus 1 sampai 1000000000000';
      }

      return null;
    },
  ),
  const SizedBox(height: 16),

  // Input tanggal untuk bagian tugas
  TextFormField(
    key: const Key('tx-date'),
    controller: _dateController,
    readOnly: true,
    onTap: _selectDate,
    decoration: const InputDecoration(
      labelText: 'Tanggal Transaksi',
      hintText: 'DD/MM/YYYY',
      prefixIcon: Icon(Icons.calendar_today),
      suffixIcon: Icon(Icons.arrow_drop_down),
      border: OutlineInputBorder(),
    ),
    validator: (value) {
      if (value == null || value.trim().isEmpty) {
        return 'Tanggal transaksi wajib diisi';
      }
```

Gambar 78. Kode langkah 16 bagian 9.

Bagian 10, baris 269 sampai 300.

Langkah 16 Bagian 10

```
      return null;
    },
  ),
  const SizedBox(height: 16),
  DropdownButtonFormField<String>(
    initialValue: _category,
    decoration: const InputDecoration(
      labelText: 'Kategori',
      border: OutlineInputBorder(),
    ),
    items: transactionCategories
        .map((v) => DropdownMenuItem(value: v, child: Text(v)))
        .toList(),
    onChanged: _saving
        ? null
        : (v) {
            setState(() => _category = v!);
            _saveDraft();
          },
    validator: (value) => transactionCategories.contains(value)
        ? null
        : 'Pilih kategori.',
  ),
  SwitchListTile(
    contentPadding: EdgeInsets.zero,
    key: const Key('tx-income'),
    title: const Text('Pemasukan'),
    subtitle: Text(_isIncome ? 'Menambah saldo' : 'Pengeluaran'),
    value: _isIncome,
    onChanged: _isTransfer || _saving
        ? null
        : (v) {
```

Gambar 79. Kode langkah 16 bagian 10.

Bagian 11, baris 301 sampai 324.

Langkah 16 Bagian 11

```
                        setState(() => _isIncome = v);
                        _saveDraft();
                      },
                ),
                if (_isTransfer)
                  const Text('Catatan transfer tidak mengubah saldo.'),
                const SizedBox(height: 32),
                ElevatedButton(
                  key: const Key('tx-save'),
                  onPressed: _saving ? null : _saveTransaction,
                  style: ElevatedButton.styleFrom(
                    backgroundColor: Colors.amber,
                    foregroundColor: Colors.white,
                    padding: const EdgeInsets.symmetric(vertical: 16),
                  ),
                  child: const Text('Simpan', style: TextStyle(fontSize: 16)),
                ),
              ],
            ),
          ),
        ),
      );
  }
}
```

Gambar 80. Kode langkah 16 bagian 11.

4.10.17 Langkah 17 Detail Transaksi

Menampilkan rincian transaksi serta menyediakan aksi edit dan hapus.

File sumber: lib/gabungan_praktikum/transaction_detail_screen.dart

Bagian 1, baris 1 sampai 32.

Langkah 17 Bagian 1

```dart
import 'package:flutter/material.dart';

import 'models/transaction_model.dart';
import 'storage.dart';

// Struktur detail dan warna dari pertemuan_04/Tugas; data constructor diverifikasi ulang berdasarkan
pemilik.
class TransactionDetailScreen extends StatefulWidget {
  final TransactionModel transaction;
  final AppStore store;
  const TransactionDetailScreen({
    super.key,
    required this.transaction,
    required this.store,
  });
  @override
  State<TransactionDetailScreen> createState() =>
      _TransactionDetailScreenState();
}

class _TransactionDetailScreenState extends State<TransactionDetailScreen> {
  late Future<TransactionModel?> _item = widget.store.transaction(
    widget.transaction.id!,
  );
  bool _busy = false;
  Future<void> _edit(TransactionModel item) async {
    await Navigator.pushNamed(context, '/add', arguments: item);
    if (mounted) {
      setState(() {
        _item = widget.store.transaction(widget.transaction.id!);
      });
    }
  }
```

Gambar 81. Kode langkah 17 bagian 1.

Bagian 2, baris 33 sampai 64.

Langkah 17 Bagian 2

```
Future<void> _delete(TransactionModel item) async {
  final confirm = await showDialog<bool>(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('Hapus transaksi?'),
      content: const Text('Transaksi ini akan dihapus.'),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context, false),
          child: const Text('Batal'),
        ),
        TextButton(
          onPressed: () => Navigator.pop(context, true),
          child: const Text('Hapus'),
        ),
      ],
    ),
  );
  if (!mounted || confirm != true) return;
  setState(() => _busy = true);
  try {
    await widget.store.deleteTransaction(item.id!);
    if (mounted) Navigator.pop(context);
  } catch (_) {
    if (mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Transaksi belum dapat dihapus.')),
      );
    }
  } finally {
    if (mounted) setState(() => _busy = false);
```

Gambar 82. Kode langkah 17 bagian 2.

Bagian 3, baris 65 sampai 96.

Langkah 17 Bagian 3

```
  }
}

@override
Widget build(BuildContext context) => Theme(
  data: ThemeData(primarySwatch: Colors.red, useMaterial3: true),
  child: Scaffold(
    appBar: AppBar(
      title: const Text('Detail Transaksi'),
      backgroundColor: Colors.red,
      foregroundColor: Colors.white,
    ),
    body: SafeArea(
      child: FutureBuilder<TransactionModel?>(
        future: _item,
        builder: (context, snapshot) {
          if (snapshot.connectionState != ConnectionState.done) {
            return const Center(child: CircularProgressIndicator());
          }
          final item = snapshot.data;
          if (snapshot.hasError || item == null) {
            return const Center(child: Text('Transaksi tidak ditemukan.'));
          }
          return SingleChildScrollView(
            padding: const EdgeInsets.all(24),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                const Icon(
                  Icons.receipt_long,
                  size: 80,
                  color: Colors.redAccent,
```

Gambar 83. Kode langkah 17 bagian 3.

Bagian 4, baris 97 sampai 128.

Langkah 17 Bagian 4

```
),
const SizedBox(height: 24),
Text(
  item.title,
  textAlign: TextAlign.center,
  style: const TextStyle(
    fontSize: 24,
    fontWeight: FontWeight.bold,
  ),
),
const SizedBox(height: 8),
Text(
  item.signedAmount,
  textAlign: TextAlign.center,
  style: TextStyle(
    fontSize: 32,
    fontWeight: FontWeight.bold,
    color: item.isTransfer
        ? Colors.blue
        : item.isIncome
        ? Colors.green
        : Colors.red,
  ),
),
const Divider(height: 40, thickness: 2),
_buildDetailRow('Kategori', item.category),
const SizedBox(height: 16),
_buildDetailRow('Tanggal', displayDate(item.date)),
const SizedBox(height: 16),
_buildDetailRow(
  'Jenis',
  item.isTransfer
```

Gambar 84. Kode langkah 17 bagian 4.

Bagian 5, baris 129 sampai 169.

Langkah 17 Bagian 5

```
                        ? 'Transfer'
                        : item.isIncome
                        ? 'Pemasukan'
                        : 'Pengeluaran',
                  ),
                  const SizedBox(height: 16),
                  _buildDetailRow('Status', 'Berhasil'),
                  if (item.isTransfer)
                    const Text('Catatan transfer tidak mengubah saldo.'),
                  const SizedBox(height: 24),
                  ElevatedButton(
                    onPressed: _busy ? null : () => _edit(item),
                    child: const Text('Edit'),
                  ),
                  TextButton(
                    onPressed: _busy ? null : () => _delete(item),
                    child: const Text('Hapus transaksi'),
                  ),
                ],
              ),
            );
          },
        ),
      ),
    ),
  );
  Widget _buildDetailRow(String label, String value) => Row(
    mainAxisAlignment: MainAxisAlignment.spaceBetween,
    children: [
      Text(label, style: const TextStyle(fontSize: 16, color: Colors.grey)),
      const SizedBox(width: 16),
      Expanded(
        child: Text(
          value,
          textAlign: TextAlign.right,
          style: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
        ),
      ),
    ],
  );
}
```

Gambar 85. Kode langkah 17 bagian 5.

4.10.18 Langkah 18 Kalkulator Kabataku

Menyediakan kalkulator operasi hitung dasar yang dapat dibuka dari aplikasi.

File sumber: lib/gabungan_praktikum/latihan/kalkulator_kabataku.dart

Bagian 1, baris 1 sampai 27.

Langkah 18 Bagian 1

```dart
import 'package:flutter/material.dart';

import '../store_scope.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: const Text('Kalkulator Kabataku')),
        body: const KalkulatorForm(),
      ),
    );
  }
}

class KalkulatorForm extends StatefulWidget {
  const KalkulatorForm({super.key});

  @override
  State<KalkulatorForm> createState() => _KalkulatorFormState();
}
```

Gambar 86. Kode langkah 18 bagian 1.

Bagian 2, baris 28 sampai 55.

Langkah 18 Bagian 2

```
class _KalkulatorFormState extends State<KalkulatorForm>
    with SavedInputs<KalkulatorForm> {
  @override
  String get draftKey => 'calculator';
  @override
  Map<String, TextEditingController> get savedControllers => {
    'first': _angka1Controller,
    'second': _angka2Controller,
    'result': _resultController,
  };
  final _formKey = GlobalKey<FormState>();
  final _angka1Controller = TextEditingController();
  final _angka2Controller = TextEditingController();

  final _resultController = TextEditingController();
  String get _hasil => _resultController.text;
  set _hasil(String value) {
    _resultController.text = value;
  }

  @override
  void dispose() {
    _angka1Controller.dispose();
    _angka2Controller.dispose();
    _resultController.dispose();
    super.dispose();
  }
```

Gambar 87. Kode langkah 18 bagian 2.

Bagian 3, baris 56 sampai 84.

Langkah 18 Bagian 3

```
void _hitung(String operator) {
  if (_formKey.currentState!.validate()) {
    double angka1 = double.parse(_angka1Controller.text);
    double angka2 = double.parse(_angka2Controller.text);
    double hasil;

    switch (operator) {
      case '+':
        hasil = angka1 + angka2;
        break;
      case '-':
        hasil = angka1 - angka2;
        break;
      case '×':
        hasil = angka1 * angka2;
        break;
      case '÷':
        if (angka2 == 0) {
          setState(() {
            _hasil = 'Tidak bisa membagi dengan 0';
          });
          return;
        }
        hasil = angka1 / angka2;
        break;
      default:
        hasil = 0;
    }
```

Gambar 88. Kode langkah 18 bagian 3.

Bagian 4, baris 85 sampai 116.

Langkah 18 Bagian 4

```
      setState(() {
        _hasil = 'Hasil : $hasil';
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    return Form(
      key: _formKey,
      child: Padding(
        padding: const EdgeInsets.all(20.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const SizedBox(height: 10),
            TextFormField(
              controller: _angka1Controller,
              keyboardType: const TextInputType.numberWithOptions(
                decimal: true,
              ),
              decoration: const InputDecoration(
                labelText: 'Angka Pertama',
                border: OutlineInputBorder(),
              ),
              validator: (value) {
                if (value == null || value.isEmpty) {
                  return 'Masukkan angka';
                }
                if (double.tryParse(value) == null) {
                  return 'Masukkan angka yang valid';
                }
```

Gambar 89. Kode langkah 18 bagian 4.

Bagian 5, baris 117 sampai 148.

Langkah 18 Bagian 5

```
        return null;
      },
    ),
    const SizedBox(height: 10),
    TextFormField(
      controller: _angka2Controller,
      keyboardType: const TextInputType.numberWithOptions(
        decimal: true,
      ),
      decoration: const InputDecoration(
        labelText: 'Angka Kedua',
        border: OutlineInputBorder(),
      ),
      validator: (value) {
        if (value == null || value.isEmpty) {
          return 'Masukkan angka';
        }
        if (double.tryParse(value) == null) {
          return 'Masukkan angka yang valid';
        }
        return null;
      },
    ),
    const SizedBox(height: 20),
    Row(
      mainAxisAlignment: MainAxisAlignment.spaceEvenly,
      children: [
        ElevatedButton(
          onPressed: () => _hitung('+'),
          child: const Text('+'),
        ),
        ElevatedButton(
```

Gambar 90. Kode langkah 18 bagian 5.

Bagian 6, baris 149 sampai 172.

Langkah 18 Bagian 6

```
                onPressed: () => _hitung('-'),
                child: const Text('-'),
              ),
              ElevatedButton(
                onPressed: () => _hitung('×'),
                child: const Text('×'),
              ),
              ElevatedButton(
                onPressed: () => _hitung('÷'),
                child: const Text('÷'),
              ),
            ],
          ),
          const SizedBox(height: 20),
          Text(
            _hasil,
            style: const TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
          ),
        ],
      ),
    ),
  );
 }
}
```

Gambar 91. Kode langkah 18 bagian 6.

BAB V
PENUTUP

5.1 Kesimpulan

Expense Tracker menggabungkan hasil praktikum menjadi aplikasi pencatatan keuangan. Form dan validasi digunakan untuk menerima input, widget dan layout membentuk tampilan, sedangkan navigasi menghubungkan halaman utama, form, dan detail transaksi. Kode gabungan ditempatkan dalam folder gabungan_praktikum, terpisah dari file latihan.

SQLite digunakan untuk menyimpan akun, transaksi, saldo awal, sesi login, dan draft. Setiap transaksi terhubung dengan akun pemiliknya. Pada pengujian Beli Buku, pengeluaran Rp 50.000 mengurangi saldo sesuai nominalnya. Transaksi dan saldo tetap tersedia setelah aplikasi ditutup paksa lalu dibuka kembali.

Analisis kode tidak menemukan masalah. Empat pengujian unit dan widget serta sembilan skenario pada emulator memberikan hasil sesuai yang diharapkan. Pengujian ini menunjukkan bahwa fungsi yang diperiksa berjalan pada lingkungan pengembangan yang digunakan.

DAFTAR PUSTAKA

[1] Nurfiah, “Praktikum I: Pengenalan Widgets: Stateless, Stateful, & Manajemen State Dasar,” dalam Modul Praktikum Aplikasi Mobile - Flutter, modul praktikum, tanpa tahun.

[2] Nurfiah, “Praktikum II: Layouting & Styling: Row, Column, Container, dan ListTile,” dalam Modul Praktikum Aplikasi Mobile - Flutter, modul praktikum, tanpa tahun.

[3] Nurfiah, “Praktikum 3: Input Widgets dan Basic Form,” dalam Modul Praktikum Aplikasi Mobile - Flutter, modul praktikum, tanpa tahun.

[4] Nurfiah, “Praktikum III: Input Widgets & Forms: Menangkap Data, Dropdown, dan Validasi,” dalam Modul Praktikum Aplikasi Mobile - Flutter, modul praktikum, tanpa tahun.

[5] Nurfiah, “Praktikum IV: Navigasi & Passing Data: Navigator, Konsep Stack, dan Constructor,” dalam Modul Praktikum Aplikasi Mobile - Flutter, modul praktikum, tanpa tahun.

[6] Nurfiah, “Praktikum V: Arsitektur Navigasi: Named Routes & Bottom Navigation Bar,” dalam Modul Praktikum Aplikasi Mobile - Flutter, modul praktikum, tanpa tahun.

LAMPIRAN

Lampiran A. Repository GitHub

https://github.com/habibiputrar/ApMob_A_2411531001_Habibi-Putra-Rizqullah/tree/gabungan-praktikum-1-5/lib/gabungan_praktikum

Lampiran B. Dokumentasi Blog Praktikum

https://hxputra.ifportofolio.com/detail-blog?slug=expense-tracker-berbasis-flutter

Lampiran C. Link Download APK

https://github.com/habibiputrar/ApMob_A_2411531001_Habibi-Putra Rizqullah/releases/download/v1.0.0/expense-tracker-v1.0.0.apk

Lampiran D. Video Penjelasan Aplikasi

https://youtu.be/rrdpuc70SCs?si=PI1FF7NwRm6LhM3c

Lampiran E. Laporan Singkat

https://drive.google.com/file/d/1ciYIMsZyw_j8VJzdShs1i9csNK5-c-wC/view?usp=sharing

Lampiran F. Link Drive Semua

https://drive.google.com/drive/folders/12m-a85i7EqdpTpGYyblydVdJppXjrbpt?usp=sharing